Liberté • Égalité • Fraternité
RÉPUBLIQUE FRANÇAISE

MINISTÈRE CHARGÉ DE L'EMPLOI

DOSSIER PROJET PROFESSIONNEL

Nom de naissance - BELLILA
Nom d'usage -
Prénom - Nabil
Adresse - [redacted]

Titre professionnel visé

Développeur Web / Web Mobile
Niveau 5

Camping Mont-Olympe

[adresse si en ligne]

MODALITÉ D'ACCÈS :

☒ Parcours de formation
☐ Validation des Acquis de l'Expérience (VAE)

Organisme de formation ► **Pôle Formation CCI Ardennes**

Lieu de formation ► **Campus SUP Ardennes - 8 Rue Claude Chrétien**
08000 - CHARLEVILLE-MEZIERES

Table des matières

L'APPLICATION 1
TABLE DES MATIÈRES 2
REMERCIEMENTS 6
RÉSUMÉ DU PROJET 7
I. Version Française 7
II. English version 8
LISTE DES COMPÉTENCES DU RÉFÉRENTIEL COUVERTES PAR LE PROJET 9
I. Front-end 9
II. Back-end 9
CAHIER DES CHARGES 10
I. INTRODUCTION 10
1. Contexte 10
2. Objectifs 10
3. Public visé 11
4. Accessibilité 11
II. FONCTIONNEMENT DU SITE 12
1. Les différents profils 12
2. Arborescence du site 12
3. Fonctionnalités en lien avec les profils 13
3.1. Liste des accès 13
3.2. Description des différentes pages 14
3.3. Scénario Utilisateur 20
3.4. Scénario Employé 21
3.5. Scénario Administrateur 23
3.5.1. Scénario de modification de paramètres 23

3.5.2. Scénario de supervision du camping 24
4. Le modèle conceptuel de données et le modèle logique 25
4.1. Modèle Conceptuel de Données (MCD) 25
4.2. Dictionnaire de Données 26
4.3. Modèle Logique de Données (MLD) 28
5. Le fichier de configuration 29
4.2. Les champs du fichier de configuration 31
LA CHARTE GRAPHIQUE 32
I. L'aspect du thème 32
II. La police de caractères 32
II.I. Policé générale 32
II.II. Policé secondaire 33
CONCEPTION CHOIX TECHNIQUES 34
I. Symfony 34
II. Doctrine 34
III. TWIG 35
IV. MYSQL 35
V. WAMP 36
VI. PHPMYADMIN 36
VII. Trello 36
VIII. GitHub 37
IX. Visual Studio Code 37
DÉVELOPPEMENT CODE 38
I. Arborescence de l'application 38
II. Développement de l'arborescence 40
1. Dossier "Assets " 40
1. Dossier " Public " 41
2. Dossier " Src " 42
3. Dossier " Template" 43

III. Modélisation de la base de données dans Symfony 45
1. Création de la base de données 45
2. Création des Entity 45
3. Migration 47
4. Création des relations 48
IV. Les outils développés 49
1. Fonctionnalités principales 49
1.1. Règles de réservation 49
1.1.1 Création de la vue 49
1.1.2 Interaction utilisateur 51
1.1.3 Création du formulaire 52
1.1.4 Traitement du formulaire 56
1.2. Réservation utilisateur 58
1.2.1. Déclencheur utilisateur 58
1.2.2. Envoie de la requête au serveur 61
1.2.3. Traitement des données sur serveur 62
1.2.4. Vérification des règles de réservations 66
1.2.5. Réponse renvoyée à l'utilisateur 71
1.3. Registre administrateur 73
1.3.1. Objectif et logique de la fonctionnalitée 73
1.4. Recherche administrateur 74
1.4.1. Objectif et logique de la fonctionnalitée 74
1.4.2. Fonctionnement du contrôleur 74
1.4.3. Visuel utilisateur 76
2. Outils 77
2.1. Visible_controller.js 77
2.1.1. Objectif et logique du contrôleur 77
2.1.2. Fonctionnement du contrôleur 77
2.1.3. Visuel utilisateur 78

2.2. Macro SVG 79
3. La page “accueil” affichée dans le navigateur 80
4. Connexion de l’utilisateur 80
5. Sécurité 83
EXEMPLE DE VEILLE **84**
CONCLUSION **88**
LEXIQUE **89**

REMERCIEMENTS

Je tiens tout d’abord à exprimer ma profonde gratitude envers mes formateurs : Steven BLOMBOU, Axel VASSAUX, Guillaume POUCET, Jean-Julien JUPINET et Valentin DE MANDAT. Leur accompagnement bienveillant a été d'une valeur inestimable tout au long de ma formation.

Je souhaite également adresser mes remerciements les plus chaleureux à M. Romain PETIT, le responsable pédagogique, ainsi qu'à toute l'équipe du pôle formation. Leur encadrement attentif a créé un environnement propice à mon apprentissage.

Je saisis cette occasion pour exprimer ma reconnaissance envers la région Grand Est et la mission locale, dont le soutien financier a rendu cette formation exceptionnelle possible.

Je voudrais également exprimer ma gratitude envers mes collègues de formation. Les moments partagés et les échanges fructueux avec vous ont enrichi mon parcours. Votre camaraderie a rendu cette expérience d'apprentissage encore plus plaisante.

Enfin, je tiens à renouveler mes remerciements à M. Romain PETIT et à l'entreprise Maîtrise Web pour m'avoir accueilli en tant que stagiaire. Leur confiance m'a permis de participer à un projet ambitieux et de progresser tant sur le plan de mes compétences de développeur que sur celui de mon expérience professionnelle.

Un grand merci à tous pour avoir contribué à mon parcours et à ma croissance professionnelle. Votre soutien a été inestimable et restera gravé dans ma mémoire.

Résumé du projet CAMPING MONT-OLYMPE

I. Version Française :

Suite à mon stage au sein de l'entreprise Maitrise Web, j'ai été chargé de la refonte complète du site internet du Camping Mont-Olympe ainsi que de son application de gestion interne.

Actuellement, le site du camping utilise le système de gestion de contenu Wordpress, tandis que son application de gestion est hébergée séparément. Cependant, cette configuration présente divers problèmes tant pour le client que pour l'équipe de développement. Parmi les principales difficultés rencontrées, on peut citer le manque de flexibilité du site en raison de sa conception sous Wordpress, une base de données archaïque avec pas moins de 86 tables, une interface administrateur peu conviviale comportant de nombreux bugs, et l'absence de possibilité pour les utilisateurs de payer en ligne.

Cette refonte permet tout d'abord d'harmoniser l'interface utilisateur et administrateur en un seul hébergement dans un projet symfony, elle améliorera aussi l'aspect visuel du site pour les utilisateurs avec des informations plus complètes et un moyen de paiement en ligne.

En outre, la refonte permettra au propriétaire et aux gérants du camping de configurer entièrement les paramètres du camping grâce à des règles flexibles. Ils pourront définir les jours d'arrivée et de départ autorisés, la durée minimale et maximale d'hébergement, le nombre maximum d'adultes autorisés… Le tout par hébergement et par saisons (périodes définies par l'administrateur). Des fonctionnalités supplémentaires seront également intégrées, telles que la possibilité de modifier les taxes, de créer des coupons de réduction, et bien plus encore.

En ce qui concerne l'application de gestion interne, les gérants pourront l'utiliser comme un outil complet de gestion. Elle leur permettra d'encaisser les clients sur place et de fournir des factures, d'organiser les réservations grâce à une validation d'arrivée et de départ, de générer un planning et bénéficier d'outils d'analyse avec des statistiques détaillées.

Bien que l'aspect e-commerce utilisateur soit essentiel, l'élément central de ce projet repose sur la conception de l'application de gestion administrateur, un véritable défi en soi. En prenant en compte le fait que le client possède plusieurs campings similaires en France, l'objectif est de rendre cette application administrateur la plus complète possible pour qu'elle puisse être utilisée facilement pour d'autres projets.

Ce projet d'envergure suscite chez moi un désir de dépassement, et je suis fier d'avoir l'opportunité de le réaliser. Je tiens à remercier chaleureusement Romain PETIT et l'équipe de Maitrise Web pour m'avoir offert cette opportunité exceptionnelle.

II. English Version :

After my internship at Maitrise Web company, I was tasked with completely redesigning the website of Camping Mont-Olympe as well as its internal management application.

As of right now, the camping website uses the Wordpress content management system, while its management application is hosted separately. However, this setup presents various issues for both the client and the development team. Some of the main challenges encountered include the lack of flexibility in the website due to its design on Wordpress, an outdated database with no fewer than 86 tables, an unfriendly admin interface with numerous bugs, and the absence of online payment options for users.

This redesign, first and foremost, aims to harmonize the user and admin interfaces into a single hosting using Symfony. It will also improve the visual aspect of the site for users by providing more comprehensive information and an online payment method.

Furthermore, the overhaul will allow the camping owner and managers to fully configure camping settings through flexible rules. They will be able to define allowed arrival and departure days, minimum and maximum accommodation duration, maximum number of adults allowed… all of that per accommodation and per seasons (periods defined by the administrator). Additional features will also be integrated, such as the ability to modify taxes, create discount coupons, and much more.

As for the internal management application, the staff will be able to use it as a complete management tool to operate the campsite. It will allow them to cash the clients and generate bills, to see the reservations and validate the check-in and check-outs, to generate a schedule and to benefit from analysis tools with detailed statistics.

While the user e-commerce aspect is crucial, the central element of this project lies in the design of the admin management application, a real challenge in itself. Considering that the client owns several similar campings in France, the goal is to make this admin application as comprehensive as possible so that it can be easily used for other projects.

This ambitious project fuels a desire for excellence in me, and I am proud to have the opportunity to carry it out. I would like to warmly thank Romain PETIT and the team at Maitrise Web for offering me this exceptional opportunity.

Liste des compétences du référentiel couvertes par le projet

I - FRONT-END

1 - Maquetter une application :

-> Arborescence, liste des fonctionnalités et maquettes du site.

2. Réaliser une interface utilisateur web statique et adaptable :

-> Site web en ligne, responsive aux différents formats desktop avec l'utilisation des Médias Query.

3. Développer une interface utilisateur web dynamique :

-> Animation et interactions utilisateurs avec JS et pseudo classes CSS

II - BACK-END

5. Créer une base de données :

-> Conception du Modèle Conceptuel de Données (MCD) et du Modèle Logique de Données (MLD) des entités et des associations à l'aide du logiciel ²JMerise.

6. Développer les composants d'accès aux données :

-> Tests unitaires, sécurité (Injections SQL).

7. Développer la partie back-end d'une application web ou web mobile :

-> CRUD, Symfony, Controllers, entités, services.

Cahier des Charges

I - Introduction

1 – Contexte :

L'application de gestion actuelle du camping mont-olympe étant limitée et séparée du site vitrine qui se base sur le CMS wordpress. Il était intéressant pour l'entreprise Maîtrise-Web ainsi que son client, le gérant du camping, de la mettre à jour.

L'application de gestion se doit d'être la plus personnalisable et indépendante possible étant donné que l'objectif est qu'elle soit livrable pour d'autres campings. L'application sera livrée au client avec une refonte entière de son site vitrine, le passant d'un site Wordpress a un site e-commerce sous Symfony 6 au design épuré.

A l'issue de mon stage chez Maîtrise-Web, on m'a proposé de me charger de cette ambitieuse mission qui sera également mon projet chef-d'oeuvre.

2 – Objectifs :

L'application web de gestion du Camping du Mont-Olympe doit
permettre la gestion interne intégrale pour le camping municipal de Charleville-Mézières, comprenant le planning et la gestion des réservations, des clients, des emplacements et leurs différentes options, des paiements et remises, de la facturation complète et de la caisse.
Elle permet également de gérer la base de données et de simplement modifier les informations du site tels que les actualités ou les emplacements.

Le site pour les utilisateurs, lui, servira de vitrine mais également d'e-commerce. Il renseignera les visiteurs sur le camping et ses produits tout en leur permettant de réserver et de payer en ligne.

3 – Le public visé :

L'application de gestion est destinée aux gérants et employés du camping du Mont-Olympe à Charleville-Mézières, et doit s'adapter avec leur mode de travail préexistant.

L'e-commerce utilisateur, lui, vise toute personne pouvant souhaiter faire du camping.

4 – Accessibilité :

L'interface e-commerce utilisateur est conçue pour s'adapter à toutes les résolutions d'écrans allant du mobile au desktop.
L'interface de gestion a été développée uniquement pour le format desktop adaptable, à différents formats d'écrans, car les gestionnaires du camping utilisent exclusivement des ordinateurs de bureau pour la gestion du camping. Ce choix limitatif a également pour raison la quantité de données sous forme de tableau à afficher.

II – Fonctionnement du site

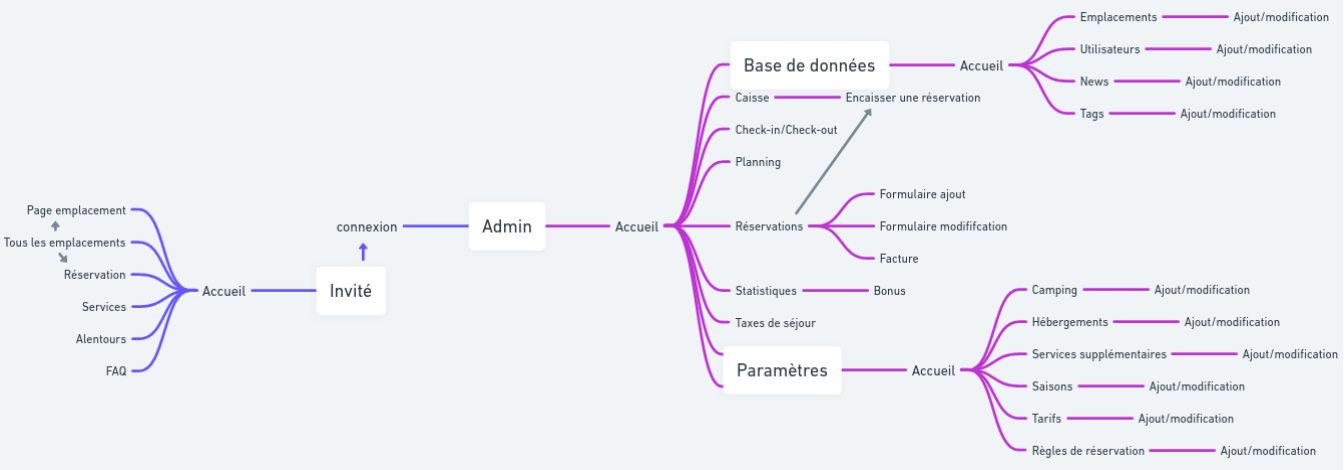
1 - Les Differents types de profil

Le projet se découpe en 3 parties en fonction du rôle de l'utilisateur.

- Un visiteur aura accès au site vitrine/e-commerce pour consulter les informations et réserver.
- Les employés du camping ont accès à l'application de gestion interne qu'ils utiliseront durant leurs services.
- Le gérant du camping et ses administrateurs attitrés auront accès aux paramètres et options du camping.

2 – Arborescence du site

Le site se compose d'un total de 41 pages.
7 pour les visiteurs du site
12 pour la gestion interne muni aux employés du camping
13 pour les paramètres et options du camping
9 pour la gestion des données



3. Fonctionnalités en lien avec les profils

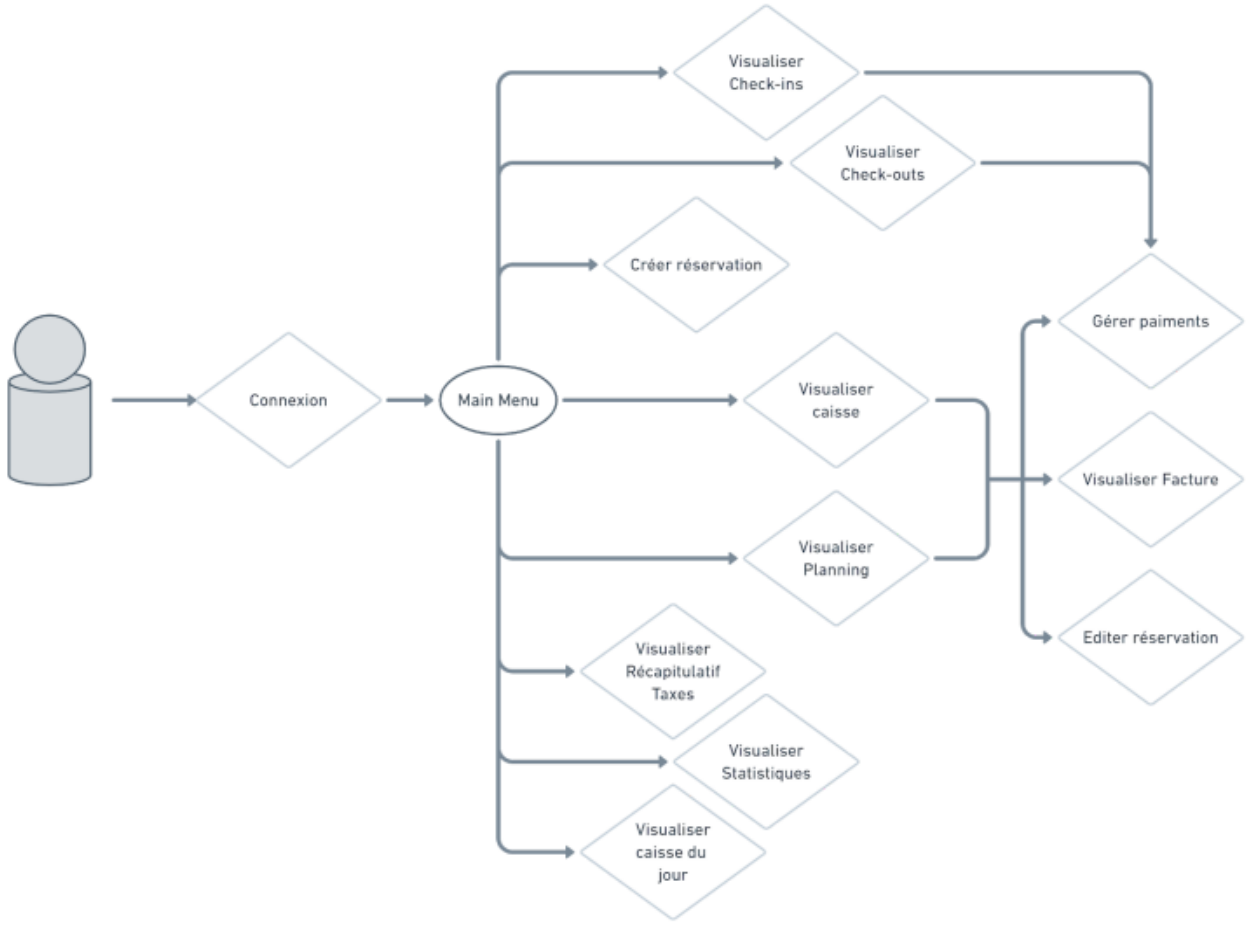
3.1 – Liste des accès

Le site n'enregistrant pas les visiteurs, le niveau d'autorisation le plus bas est pour les employés et la gestion interne.
Un rôle permet en plus de gérer les paramètres et options, un autre de visionner le registre des actions entreprises et un dernier d'attribuer des rôles.

```yaml
role_hierarchy:
  ROLE_ADMIN:
  ROLE_SETTINGS: [ROLE_ADMIN, ROLE_ALLOWED_TO_CHANGE_SETTINGS]
  ROLE_LOGS: [ROLE_SETTINGS, ROLE_ALLOWED_TO_SEE_LOGS]
  ROLE_SUPER_ADMIN: [ROLE_LOGS, ROLE_ALLOWED_TO_CREATE_ADMINS]

access_control:
  - { path: ^/admin/users, roles: ROLE_ALLOWED_TO_CREATE_ADMINS }
  - { path: ^/admin/logs, roles: ROLE_ALLOWED_TO_SEE_LOGS }
  - { path: ^/admin/settings, roles: ROLE_ALLOWED_TO_CHANGE_SETTINGS }
  - { path: ^/admin, roles: ROLE_ADMIN }
```

Diagramme de cas d'utilisation d'un employé durant son service



3.2 – Description non exhaustive des différentes pages

Pages utilisateurs, pages gestionnaire, pages paramètres, pages super admin

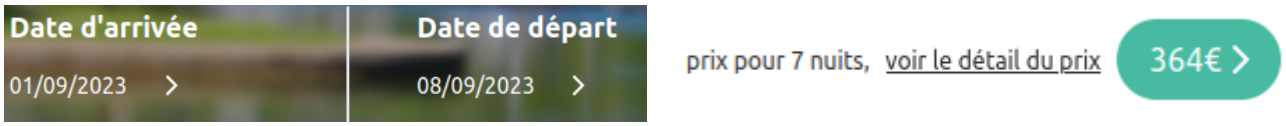
Page d'accueil - Header avec CTA réservation, sections résumé des autres pages au défilement, footer avec contact



Page tous les emplacements - Liste des emplacements avec aperçu des informations

Page emplacement - Visuel de l'emplacement correspondant avec des informations complémentaires (heure de check-in, nombre total d'emplacements, emplacements dispo, tarifs, numéro des emplacements)

Page réservation - Permet au visiteur de réserver en choisissant une date de départ et une date de fin, les emplacements disponibles lui sont montrés avec un calcul du prix en fonction de la durée du séjour et du nombre de personnes.



Page services - Une page décrivant les services qu'offre le camping (animaux, wifi..)

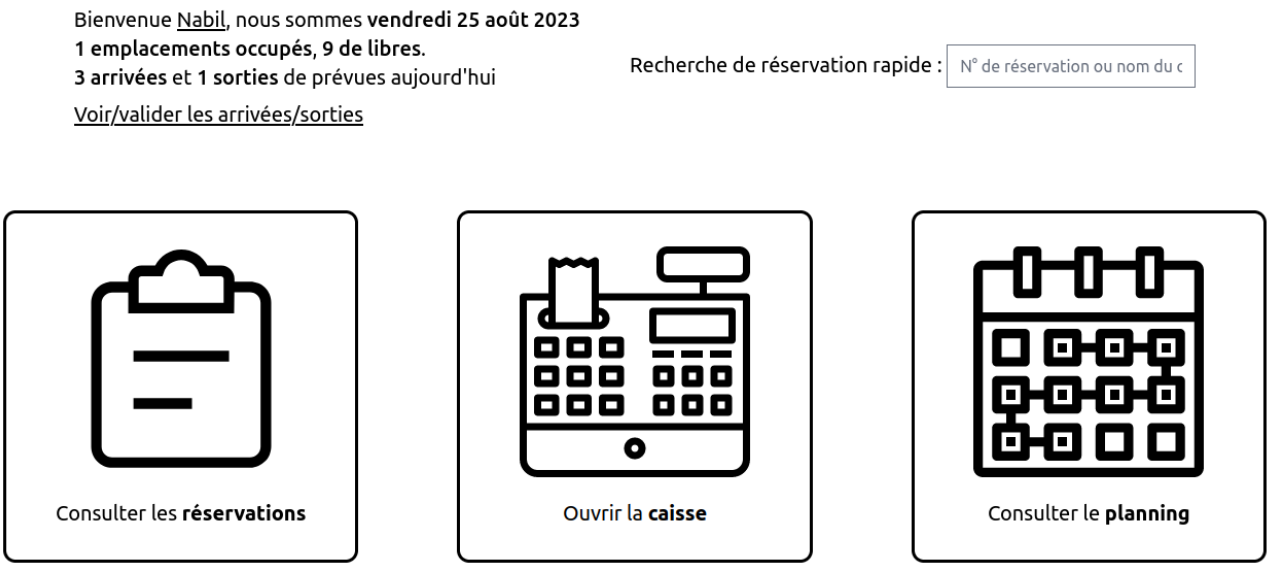
Page alentours - Une page proposant des activités ou endroits à visiter aux alentours

Page FAQ - Une page répondant aux questions fréquemment posées



Page connexion - Une interface de connexion sans inscription, permettant l'accès uniquement à l'équipe du camping.

Page d'accueil gestion interne - L'interface d'accueil pour les gestionnaires, offrant des informations sur le service actuel (nombre d'arrivées et sorties prévues aujourd'hui, emplacements libres/occupés, recherche rapide d'une réservation) et leur permettant un accès simple et rapide aux éléments importants tel que la caisse.



Caisse - Affiche la caisse du jour avec un récapitulatif des paiements, permet de consulter la caisse d'un autre jour ou d'un mois entier. Permet aussi de trouver une réservation pour l'encaisser

Check-in/Check-out - Liste des réservations en cours, ainsi que les arrivées et départs attendus ce jour. Ceux-ci peuvent être validés lorsqu'ils auront été confirmés. La page est réglée de base sur la date du jour, mais il est possible de consulter d'autres jours

Planning - Un planning consultable par mois ou semaine, pour un emplacement ou tous. Les informations des réservations affichées sont directement consultables avec une modal.

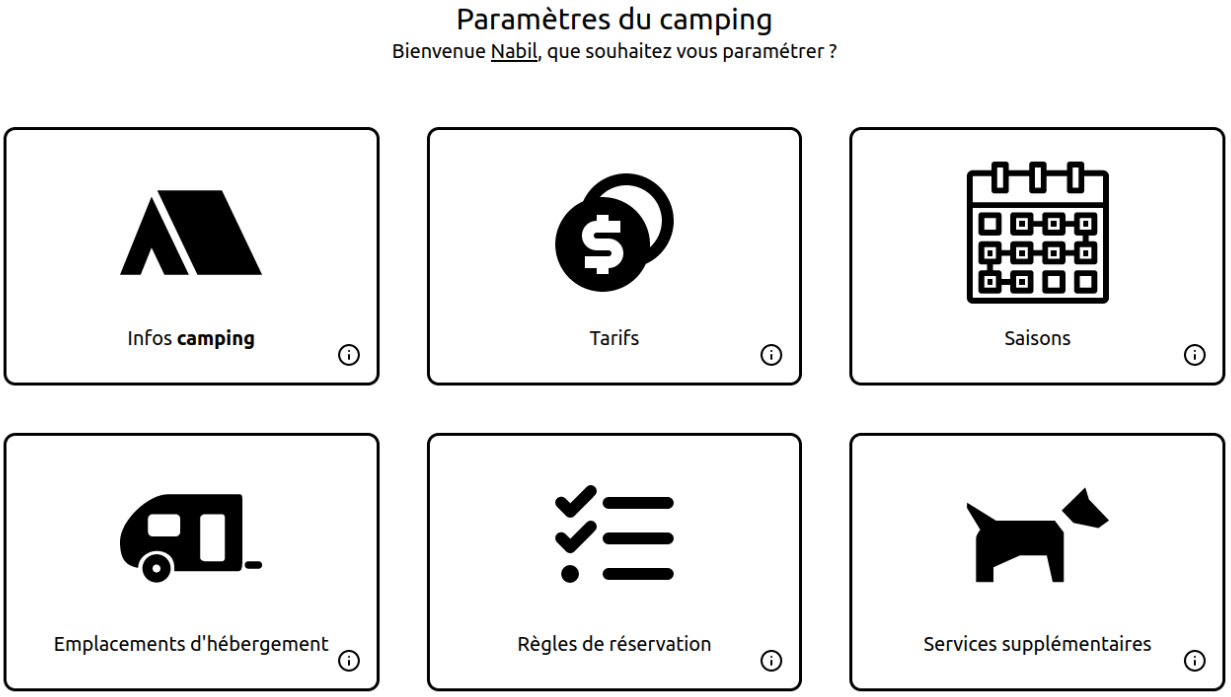
Statistiques - Une page avec divers statistiques

Taxes de séjour - Permet de générer les taxes de séjour pour la période choisie

Réservations - Un tableau des réservations avec possibilité de rechercher depuis le numéro ou le nom du client. Chaque réservation est modifiable et supprimable, l'employé peut également lui générer une facture ou encaisser un paiement. Le gestionnaire peut également créer une réservation sur place.

ID	Check-in	Check-out	Type d'hébergement	Emplacement	Nombre	Infos client	Prix	Date de réservation	Actions
1	10/08/2023	20/08/2023	Camping Car	50	Adultes :2 Enfants :1	Voir les informations du client	Tarif : 99.99 Payé : 0.00	August 11, 2023 12:51	Modifier Supprimer Encaisser Facture
2	20/08/2023	26/08/2023	Camping Car	50	Adultes :2 Enfants :1	Voir les informations du client	Tarif : 99.99 Payé : 0.00	August 11, 2023 12:51	Modifier Supprimer Encaisser Facture

Page d'accueil paramètres - Permet d'accéder aux paramètres disponibles



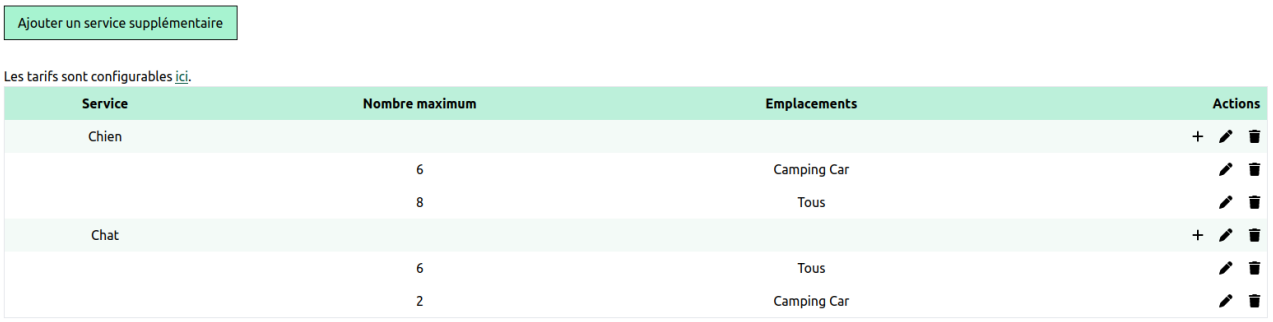
Camping - Permet de consulter et modifier les informations du camping tel que le numéro de téléphone ou la carte. Ces informations sont présentes sur le site ainsi que sur les factures générées.

Emplacements - Permet de créer, visualiser, modifier et accéder aux emplacements.



ID	Nom	Description	Disponible	Emplacements	Actions
1	Camping Car	Notre Camping propose des forfaits en emplacement camping-car et se fera un plaisir de vous accueillir chaleureusement toute l'année.	oui	5	
2	Caravane	49, c'est le nombre d'emplacement confort caravane que dispose le camping du mont-olympe	oui	4	
7	Port	Super le port !	oui	1	
12	Test	Un type d'hébérgement supplémentaire	non	0	

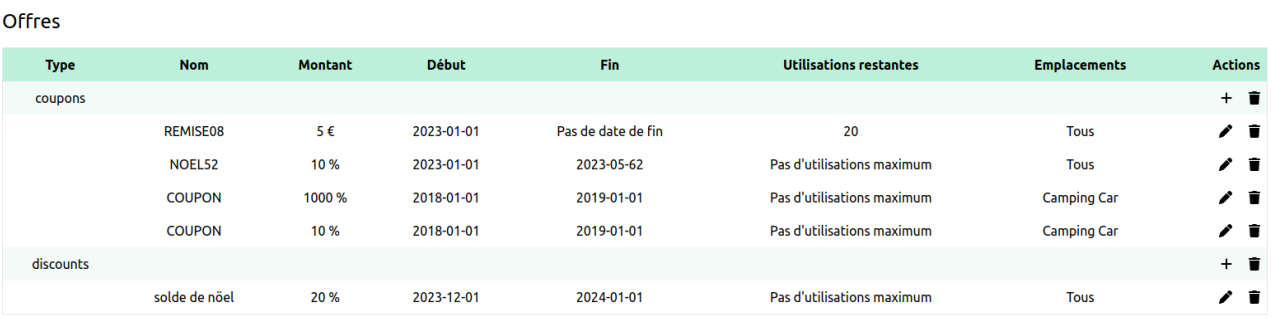
Services supplémentaires - Permet de créer, visualiser, modifier et accéder aux services supplémentaires.



Tarifs - Permet d'ajouter des règles de tarifs pour les emplacements, les services et les catégories d'âge. Un tarif se définit par un prix, en choisissant si le prix se calcule par nuit et/ou par personne et par les saisons durant lesquelles la règle prend effet. Il y a également possibilité d'ajouter des offres et des coupons, en pourcentage ou en euro, avec une date de fin ou un nombre d'utilisations.
Toutes les règles sont modifiables et supprimables.







Saisons - Permet d’ajouter des saisons, et des périodes pour ces saisons avec une date de début et de fin, ainsi que les jours concernés.
Les saisons ainsi que les règles sont modifiables et supprimables.



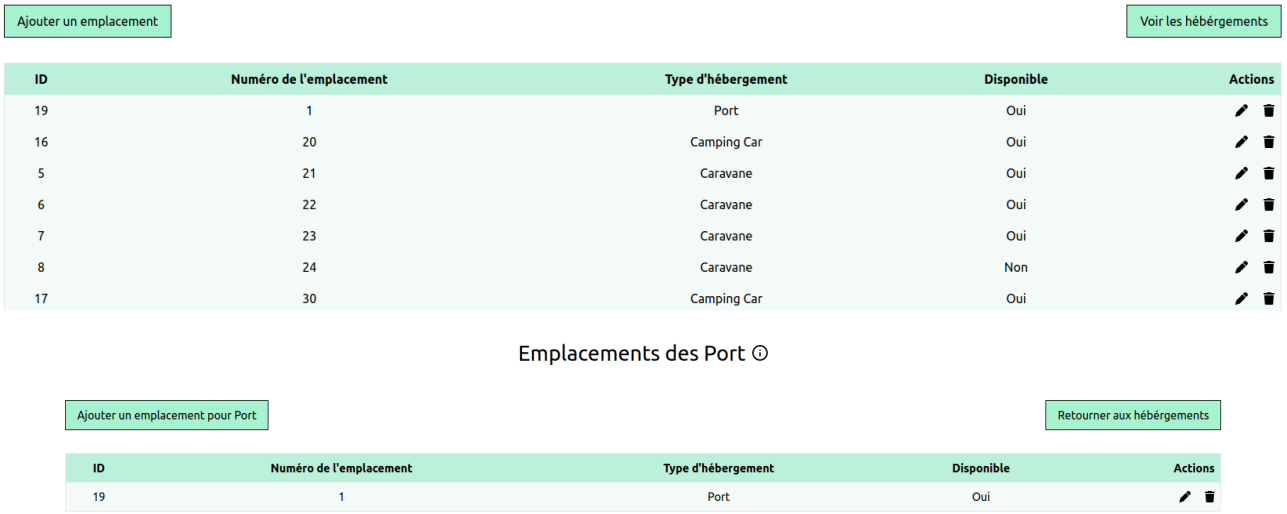


Règles de réservation - Permet d’ajouter des règles de réservation pour les jours d’arrivées/de sorties et la durée minimale/maximale autorisées. Ces règles se définissent par type d’emplacements et par saisons et sont toutes modifiables et supprimables.



Page d’accueil base de données - Permet d’accéder aux tables de données.

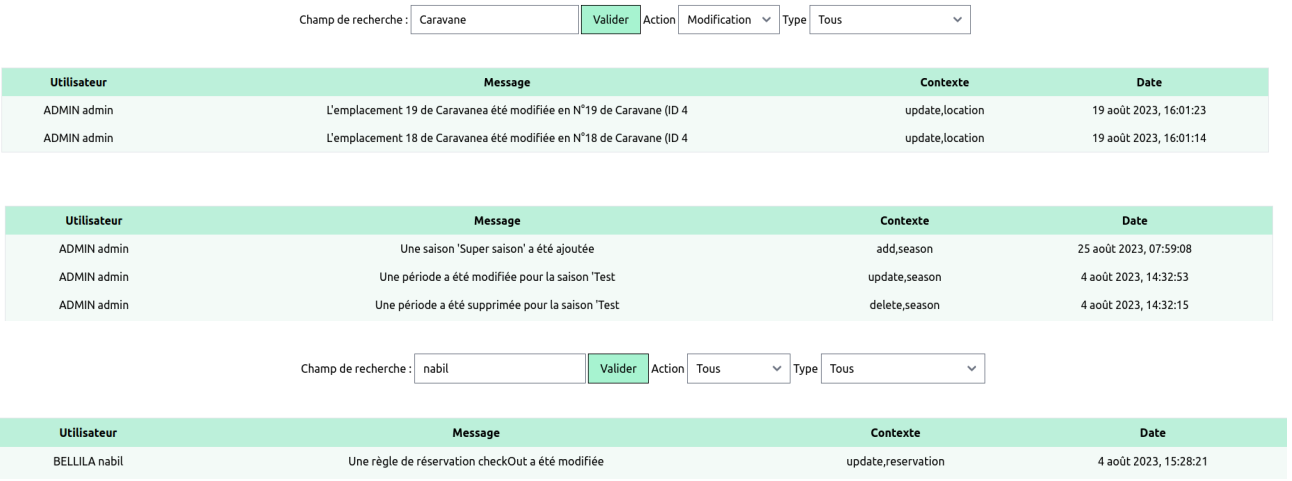
Emplacements - Permet de créer, visualiser, modifier et accéder aux emplacements. Possible pour un type d’hébergement ou tous.



Utilisateurs - Permet de créer un nouvel utilisateur, de visualiser ceux existants ainsi qu'un modal de leurs dernières actions sur l’application.

Actualitées - Permet de modifier les actualitées qui sont affichés en haut de la page d’accueil du site

Registre - Permet de visualiser toutes les actions entreprises dans l’interface de gestion et d’administration, ainsi que l’auteur associé et des tags pour faciliter la recherche par filtre. Des commentaires peuvent également être ajoutés afin de communiquer entre équipiers si besoin.



3.3 – Scénario utilisateur

Scénario de première visite

Le visiteur arrive sur le site en espérant avoir trouvé un endroit où camper, il découvre au scroll des sections d'informations répondant à la plupart de ses questions.

Le visiteur est incertain du type d'emplacement qu'il souhaite, il se rend donc sur la page des emplacements avant de finalement choisir un camping-car pour sa famille. A l'issue de quelques minutes de navigation, il se décide à choisir cet endroit pour sa semaine de vacance.

Il quitte son ordinateur de bureau et descends dans le salon avec son téléphone afin de montrer le camping à sa femme.

Sa femme se demande s'ils peuvent être accompagnés de leur chien. Pour répondre à cette question, le visiteur souhaite contacter le camping à l'aide du formulaire de bas de page. Il est cependant stoppé par un message lui demandant de consulter la foire aux questions.
Il y trouve réponse à sa question et se décide donc enfin à réserver en utilisant le bouton l'ayant suivi durant toute sa navigation.

Il définit sa date d'arrivée ainsi que sa date de départ et regarde les résultats : mince, le camping-car n'y figure pas. Aucun emplacement n'est disponible durant cette période.
Le visiteur est informé par le système de la prochaine disponibilité, celle-ci est 2 jours plus tard, le visiteur décale donc son arrivée.
Il choisit ses options et valide sa réservation avant de choisir la méthode de paiement, il choisit de payer sur place.

Il consulte sa boîte mail et remarque la présence d'un email de confirmation lui récapitulant toutes les informations nécessaires.

Il éteint son ordinateur et annonce la bonne nouvelle à son enfant.

3.4 – Scénario employé

Scénario employé de gestion interne

Il est 14h30, Martine démarre son service. Elle se place à l'accueil et allume l'ordinateur de bureau. Elle arrive sur la page de connexion et rentre ses identifiants.

Martine est redirigée vers l'interface de gestion interne et peut démarrer son service, elle est informée qu'une arrivée ainsi qu'un départ sont attendus aujourd'hui.

Un client appelle au téléphone et souhaite réserver un emplacement camping car du 1er au 13 juin. Martine clique sur "créer une réservation" :
Elle est redirigée vers le formulaire de création de réservation. Elle entre le nombre d'adultes et d'enfants, ainsi que la plage de dates, et valide ensuite pour passer à l'étape 2. Elle choisit ensuite l'emplacement et diverses options souhaitées. Elle valide et passe à l'étape finale en rentrant les informations du client.

Une demi-heure passe, le même client rappelle. Il ne souhaite plus venir du 1er au 13 juin mais du 1er au 10 juin.
Marine clique sur "Consulter les réservations" et accède directement aux dernières réservations effectuées. Elle clique sur "éditer la réservation" afin de modifier la date de départ.

Un client arrive à l'accueil et demande à régler son séjour :
Marine a plusieurs options pour accéder à la réservation correspondante, elle décide d'utiliser la barre de recherche de la page d'accueil en entrant le nom du client. Elle est redirigée vers la page de la réservation où elle clique sur "encaisser un paiement".

Le client paye la totalité du montant en espèces : Marine choisit dans le formulaire le moyen de paiement souhaité, indique le montant à régler, et la date courante. Elle clique ensuite sur "indiquer le règlement". Le règlement est enregistré.
Le client souhaite également une facture, Marine clique sur "générer une facture".

Un autre client arrive à l'accueil et dit avoir réservé l'emplacement 12 pour la semaine.
Pour accéder à la réservation, cette fois Marine clique sur "check-in/check-out" pour vérifier que le client est bien censé arriver aujourd'hui. Elle confirme les informations et remarque que le client a déjà réglé le paiement en ligne. Elle confirme l'arrivée et laisse le client rentrer.

C'est la fin de journée, Martine clique sur "Ouvrir la caisse" et imprime le récapitulatif de la journée pour le passer à la comptabilité.

Elle se déconnecte pour laisser place au prochain employé, mais avant elle consulte le planning afin de savoir à quoi s'attendre les jours à venir.
Elle se rend compte qu'elle devra s'attendre à deux nouvelles arrivées demain.

3.5 – Scénario administrateur

3.5.1 – Scénario de modification de paramètres

Franck, le gérant du camping, organise une refonte de son camping. Il souhaite modifier le logo du camping, certaines règles de réservations ainsi que certains tarifs.

Franck se rend sur son site et se connecte, il arrive sur la page d'accueil de la gestion inter. Il est enregistré en tant que super admin et à donc tous les accès. Il clique sur "Paramètres" et accède aux options du camping.

Il commence par cliquer sur la section "camping" et modifie le logo en téléchargeant une nouvelle image.

Franck retourne à l'accueil puis clique sur "règle de réservations", il ajoute une règle d'arrivée pour le port, son nouvel emplacement, et interdit le mardi afin de le réserver pour l'entretien des emplacements non occupés.
Il ajoute également une règle de durée minimum de séjour de 2 nuits afin de ne pas gaspiller inutilement la préparation de l'emplacement.

Franck souhaite augmenter les prix durant l'été, il clique donc sur "saison" et créer une nouvelle saison "Haute Saison 2023", il lui attribue une période du 01 juin au 31 août 2023 pour tous les jours de la semaine.
Franck supprime également les anciennes saisons qui n'ont plus d'intérêt.

Franck doit maintenant modifier les tarifs à l'aide de cette nouvelle saison, il retourne à l'accueil et clique sur "Tarif". Il a accès aux tarifs des emplacements, services, et tranche d'âge. Pour chacun d'entre eux, il ajoute une nouvelle règle en sélectionnant la saison "Haute Saison 2023" qu'il vient de créer.

Pour ce nouveau départ Franck souhaite donner un coup de boost à son camping, il souhaite donc créer un bon de réduction pour accompagner sa campagne publicitaire. Sur la même page, il clique sur "ajouter un coupon" et ajoute le coupon "été2023" durant offrant 10% de réduction sur son nouvel emplacement : le port

3.5.1 – Scénario de supervision du camping

C’est la fin du mois et Franck, le gérant du camping, souhaite surveiller l’activité de son camping.
Franck se rend sur son site et se connecte, il arrive sur la page d’accueil de la gestion interne. Il est enregistré en tant que super admin et à donc tous les accès.

Il clique sur ‘Registre’ et accède à la liste de tous les événements ayant eu lieu. Il souhaite vérifier que ses employés respectent les horaires.
Il clique donc sur les filtres et sélectionne ‘connexion’ et ‘déconnexion’. Il aperçoit donc toutes les dernières connexions/déconnexions ayant eu lieu.

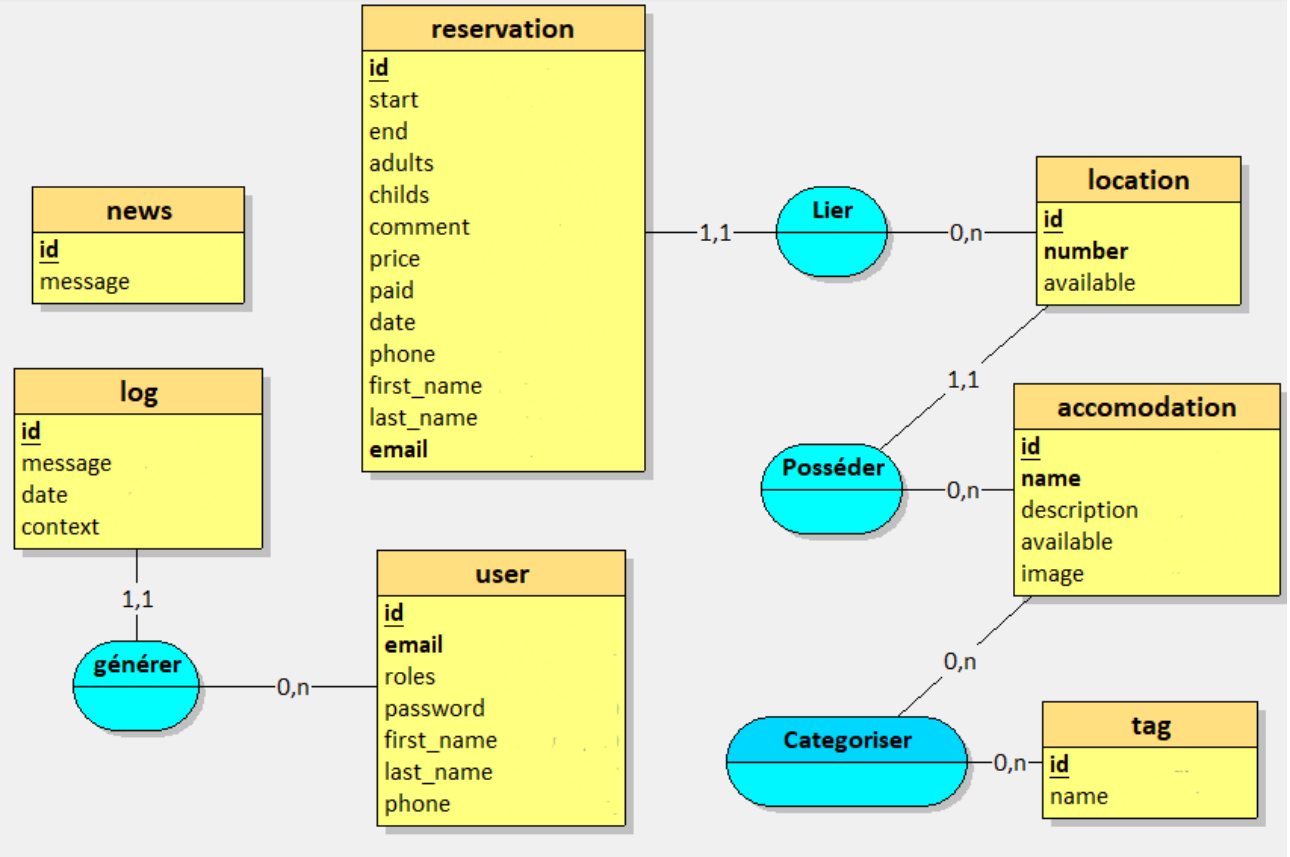
Franck souhaite analyser l’avancée de son camping, il commence donc par ouvrir la caisse et consulter celle du mois. Il observe le récapitulatif des paiements et est satisfait, il se rend également compte que de moins en moins de paiements se déroulent en espèces.
Pour vérifier cette impression, il décide de consulter les statistiques.

Enfin, Franck retourne à l’accueil puis consulte le planning. Il souhaite voir la différence entre les réservations de l’été dernier et de cet été, afin de voir si sa campagne de publicité a porté ses fruits.

Franck a pu avoir toutes les informations qu’il souhaitait, il se déconnecte et part se coucher.

4 - Les Modèles Conceptuel de Données et le Modèles Logique

4.1 – Modèle Conceptuel de Données



Les Tables :

- L'élément central du modèle est la table **Réservation**, qui représente une réservation.
- La table **Location** représente un emplacement du camping.
- La table **Accommodation** représente un type d'hébergement (Caravane, emplacement simple..)
- La table **Tag** représente un tag (ensoleillé, accès handicapé..)
- L'entité **User** représente un utilisateur et sert pour l'accès à l'application administrative
- L'entité **Log** représente un message stocké dans un registre, automatiquement généré lorsqu'une action administrateur est entreprise.
- L'Entité **News** représente une actualité affichée sur le site

Les Associations (Justification des Cardinalités)

- Une **Réservation** doit avoir un et un seul **Emplacement**. Un **Emplacement** peut avoir une infinité de **Réservation.**

- Un **Emplacement** doit avoir un et un seul **Type d'hébergement**. Un **Type d'hébergement** peut avoir une infinité d'**Emplacement**.

- Un **Type d'hébergement** peut avoir une infinité de **Tag**. Un **Tag** peut avoir une infinité de **Type d'hébergement**.

- Un **Message de Registre** doit avoir un et un seul **Utilisateur**. Un **Utilisateur** peut avoir une infinité de **Message de Registre.**

4.2 – Dictionnaire de Données

Réservation

Id	Nom conceptuel	Nom logique	Type
🔑	id		COUNTER
	start	start_date	DATE
	end	end_date	DATE
	adults	adults_count	INT
	childs	childs_count	INT
	comment		VARCHAR(510)
	price		DECIMAL(6,2)
	paid		DECIMAL(6,2)
	date	reservation_date	DATE
	phone		VARCHAR(255)
	first_name		VARCHAR(255)
	last_name		VARCHAR(255)
	email		TIME

Location

Id	Nom conceptuel	Nom logique	Type
🔑	id		COUNTER
	number		INT
	available		LOGICAL

Accommodation

Id	Nom conceptuel	Nom logique	Type
🔑	id		COUNTER
	name		VARCHAR(255)
	description		TEXT
	available		LOGICAL
	image		VARCHAR(255)

Tag

Id	Nom conceptuel	Nom logique	Type
🔑	id		COUNTER
	name		VARCHAR(255)

User

Id	Nom conceptuel	Nom logique	Type
🔑	id		COUNTER
	email		VARCHAR(255)
	roles		json
	password		VARCHAR(255)
	first_name		VARCHAR(255)
	last_name		VARCHAR(255)
	phone		VARCHAR(255)

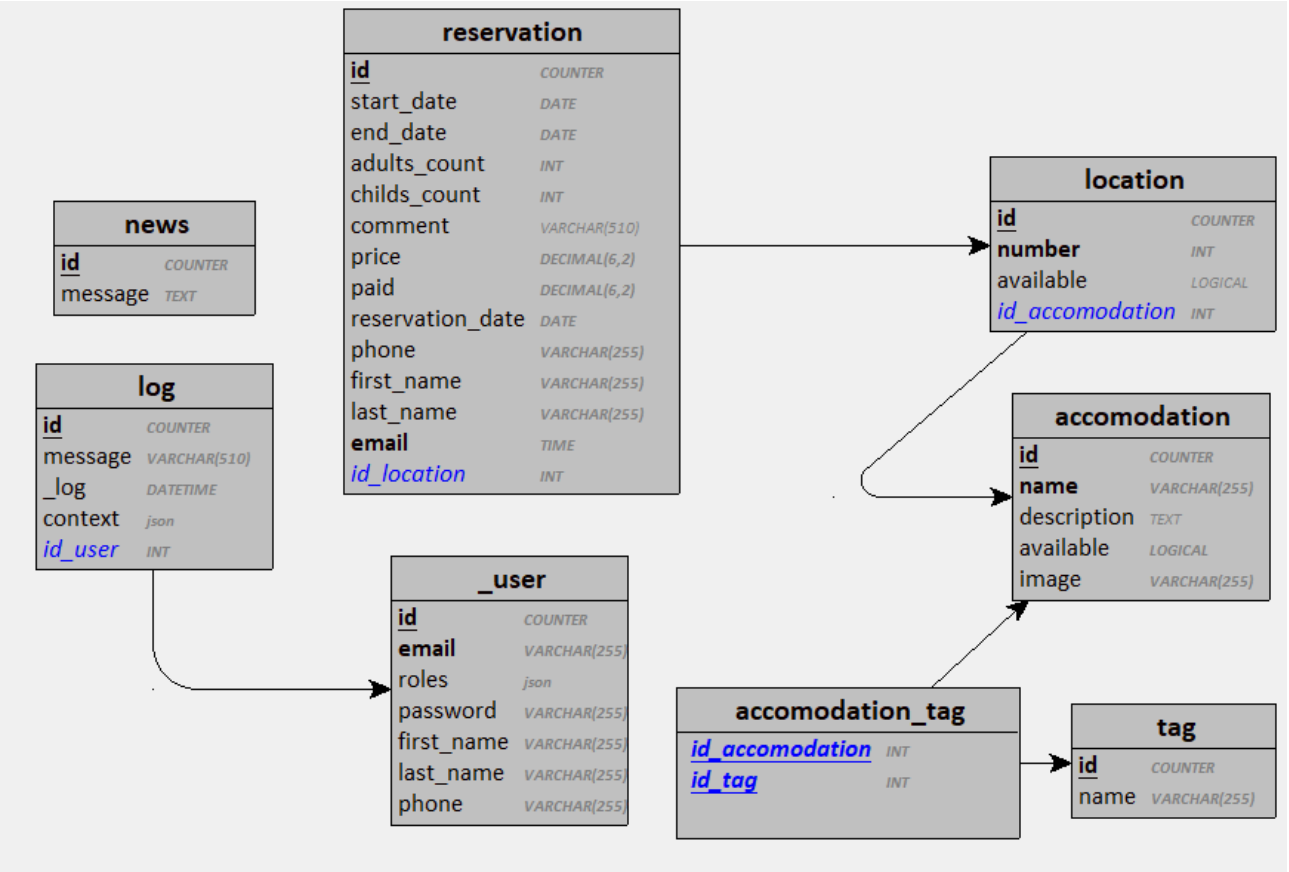
Log

Id	Nom conceptuel	Nom logique	Type
🔑	id		COUNTER
	message		VARCHAR(510)
	date	log_date	DATETIME
	context		json

News

Id	Nom conceptuel	Nom logique	Type
🔑	id		COUNTER
	message		TEXT

4.3 – Modèle Logique de Données



Une seule entité est créée lors du passage du MCD au MLD, car nous n'avons qu'une relation du type n-n :

La table **accomodation_tag** ferra le lien entre **accomodation** et **tag** en contenant des paires de clés 'accomodation' et 'tag'

La table **Location** récupère un id_**accomodation** tandis que la table **Reservation** récupère un id_**location.**

Par extension, la table réservation a donc un lien indirect avec un hébergement et avec ses tags.

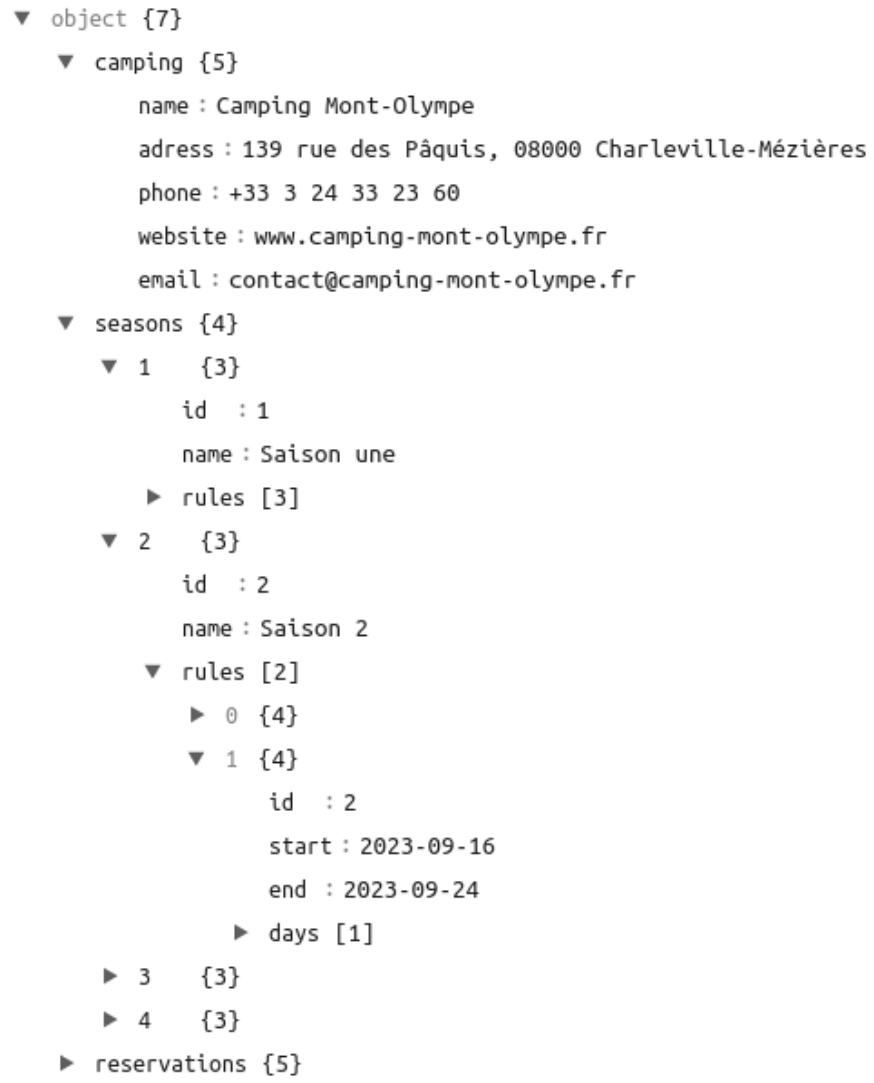
5 - Le fichier de configuration

Pour gérer les paramètres du camping, un fichier config.json est utilisé.

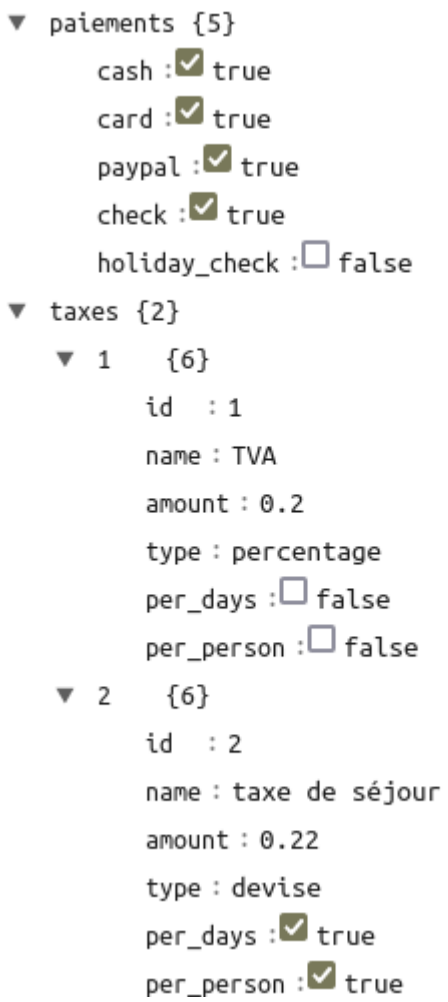
Le json contient quelques données mais majoritairement des règles et options.



Exemple avec le champ ‘saisons’ : le champ contient plusieurs saisons, contenant elles une ou plusieurs règles définissant leurs portées.



Exemple avec les champs paiements' et 'taxes' qui permettent respectivement d'activer ou désactiver des modes de paiements, et de gérer les taxes s'appliquant durant le paiement :



Le fichier config.json ce situe dans le dossier 'config' de l'architecture symfony, il n'est accessible que par un service ConfigService dont l'accès est restreint.

5.2 – Les champs du fichier de configuration

Le fichier config.json ce situe dans le dossier ‘config’ de l’architecture symfony, il n’est accessible que par un service ConfigService dont l’accès est restreint.

Camping : A obligatoirement ces 5 champs prédéfinies qui possèdent chacun une valeur personnalisable

Saisons : N’a pas obligatoirement de champ et peut en avoir plusieurs, chaque saison n’a pas obligatoirement de règles et peut en avoir plusieurs

Réservations : A obligatoirement ces 5 champs de règles de réservations prédéfinies, chaque règle de réservation n’a pas obligatoirement de champs et peut en avoir plusieurs. Ces champs sont définis par la base de données.

Chaque champ des règles de réservations n’a pas obligatoirement de règles et peut en avoir plusieurs.

Tarifs : A obligatoirement ces 4 champs de catégories prédéfinies, chaque catégorie n’a pas obligatoirement de champs et peut en avoir plusieurs. Chaque champ n’a pas obligatoirement de règles et peut en avoir plusieurs.

Services supplémentaires : N’a pas obligatoirement de champ et peut en avoir plusieurs, chacun de ses champs n’a pas obligatoirement de règles et peut en avoir plusieurs.

Paiement : A obligatoirement ces 5 champs prédéfinies qui possèdent chacun une valeur personnalisable

Taxes : N’a pas obligatoirement de champ et peut en avoir plusieurs.

Chaque règle a obligatoirement un champ et ne peut en avoir qu’un.

La charte graphique

I. L’aspect du thème

Le projet étant en rapport avec la nature, je suis d’abord parti sur une base de vert. Cela ne convenait pas parfaitement et j’ai donc opté pour une variante se rapprochant du bleu.

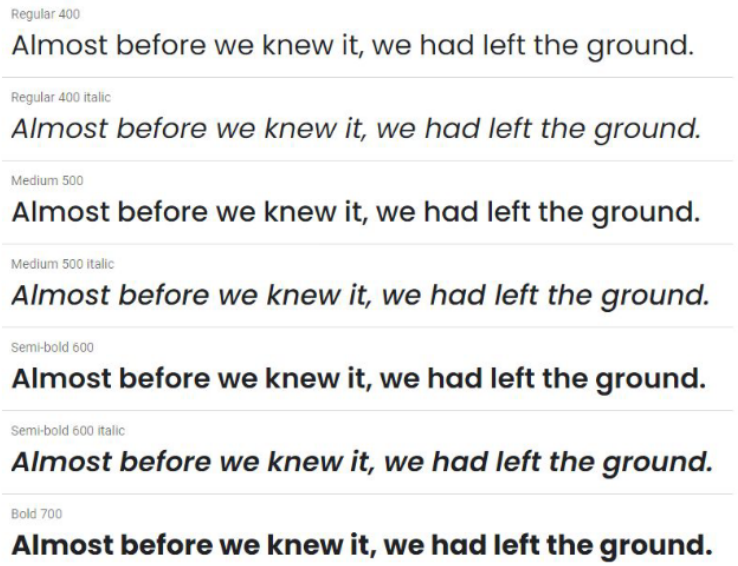
J’ai généré une palette à partir du Lochinvar #2E9E89 à l’aide du site https://uicolors.app que j’ai ensuite intégré à ma configuration tailwind.



II. La police de caractère

II.I - Police générale

Pour l’interface administrateur j’ai choisi une police simple et lisible, adaptée au travail quotidien : **Poppins**



II.I - Police secondaire

Pour le site e-commerce j'ai choisi d'utiliser comme police secondaire **Pacifico.** Cette police a pour but de mettre en valeur certains éléments tout en donnant un effet naturel se mariant avec la verdure des images.

Pacifico est raffinée et créative, apportant une certaine fraîcheur au visuel. Cependant, étant une police de type 'fantaisiste' elle est utilisée avec parcimonie.



Conception et choix techniques

I - SYMFONY



Dans l'objectif de conserver et d'améliorer le site, je me suis lancé dans l'utilisation de Symfony. Je l'ai choisis car c'est un framework francais avec une communautée active et une documentation complète. Et il permet fournir une structure stable, maintenable et une bonne gestion de la sécurité. De plus, cela me permet d'apprendre un nouveau framework et à développer un site dans le Modèle MVC (Model Vue Controller).

II – DOCTRINE



Doctrine est un Object Relational Mapper ou ORM (couche d'abstraction à la base de données) pour PHP. Doctrine est l'ORM par défaut du framework Symfony mais Il permet la persistance des objets PHP ainsi que de faire le lien entre les objets et les éléments contenues dans la base de données. Il est également très documenté.

III – TWIG



Le paquet Twig est intégré de base à Symfony comme moteur de template. C'est un outil d'intégration de la logique dans les vues et qui permet de transmettre ou recevoir des données avec le contrôleur. Tout comme doctrine, la plupart des documentations symfony se base dessus, le choix était donc évident.

IV – MySQL



La base de données tourne sur un serveur MySQL de base de données relationnelles. MySQL est un SGBD ou un système de gestion de bases de données relationnelles. Il permet d'interagir avec une base de données avec des requêtes SQL. On interagit avec lui par PHP, par lignes de commande ou par une application web de gestion de SGBD comme phpMyAdmin, HeidiSQL ou MySQL Workbench.

V - SERVEUR PHP : WAMP

Solution tout-en-un pour windows, intégrant un serveur web Apache, PHP, un serveur mySQL, PHPmyAdmin.

VI - PHPMYADMIN

PhpMyAdmin est l'interface d'administration pour interagir avec MySQL. Elle permet de visualiser la bonne implémentation de la base de données ainsi que le bon déroulement des opérations réalisées.

VII - TRELLO

Il m'a permis de visualiser les différentes étapes du projet et de mettre en place une méthode agile, en divisant mon projet en plusieurs sous-projets.

VIII - GITHUB

GitHub est un outil de dépôt de sources permettant un versionning et une mise en commun de code efficace. C'est un outil dont je ne pourrais plus me passer même sur du travail personnel tant il est utile à l'organisation d'un projet.

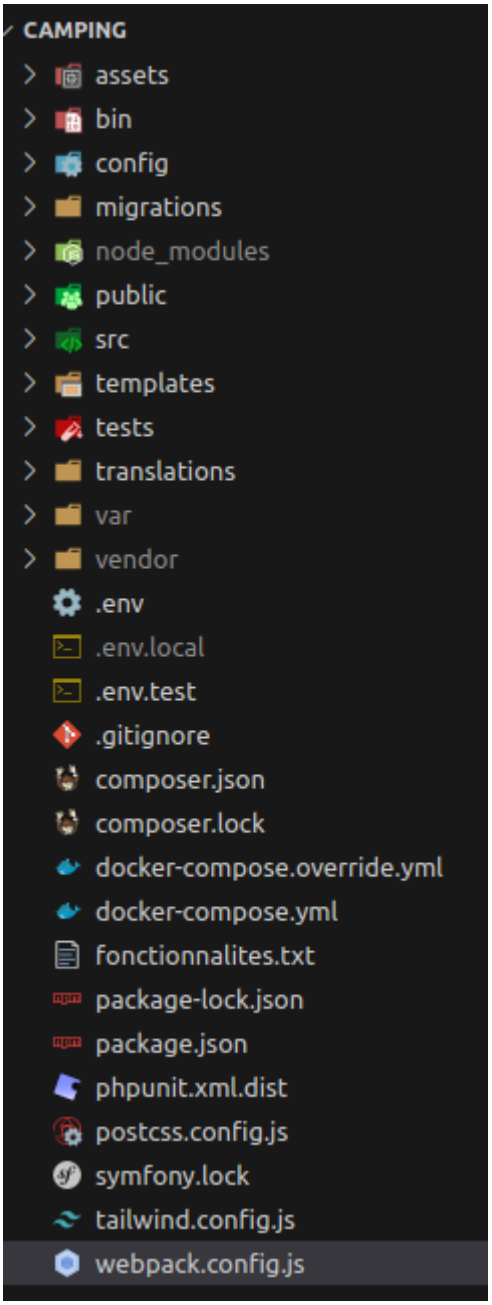
XII - VISUAL STUDIO CODE

Visual Studio Code

J'ai utilisé l'IDE Visual Studio Code pour mon projet. Il m'a permis de travailler sous l'invite de commande directement dans l'éditeur et dispose d'un module d'autocompletion ainsi que de nombreuses extensions comme Beautify, Live server ou encore ….(voir extension utilisée).

Développement code

I - Arborescence principale du site



Racine :
On y trouve les fichiers ".env" dans lequel sont renseignées les variables d'environnement (utilisé par exemple pour la connexion à la BDD).

Puis les différents fichiers de dépendances tels que composer.json, package.json, symfony.lock (les fichiers utilisés par composer et npm).

Un fichier .gitignore utilisé par git pour indiquer les fichiers et dossiers à ignorer dans le dépôt de source.

Un fichier tailwind.config.js pour gérer le framework CSS tailwind

Un fichier webpack.config.js pour gérer le bundle webpack.encore utilisé principalement pour compiler les assets

Assets :
On y trouve les fichier css et js qui sont ensuite compilé dans le dossier public

Bin :
C'est ici que se trouve les exécutables de Symfony, comme le fichier console.
Les fichiers dans ce dossier seront donc exécutables en ligne de commande.

Config :
Contient les fichiers de configuration du framework et des différents packages. Notamment le fichier "security.yaml" qui gère la connexion utilisateur et la sécurité du site. J'y stocke également le fichier de paramètres du camping 'config.json'

Migrations :
Contient l'intégralité des fichiers de versionning de base de données et l'historique des changements effectués. Ce sont en fait des fichiers qui exécutent des requêtes SQL permettant de passer le modèle de base de données d'une version à une autre au long du projet.

Node_modules :
Contient les packages et dépendances externes créées avec npm et utilisés par l'application, comme par exemple le bundle select2 ou les fichiers nécessaires à l'intégration de tailwind ou fontawesome.

Public :
Ceci est le dossier d'entrée de notre application, mais aussi le dossier public, nous allons y mettre tous les fichiers accessibles au public. Il contient notamment le contrôleur frontal de Symfony, le css et js compilé et les images du site.

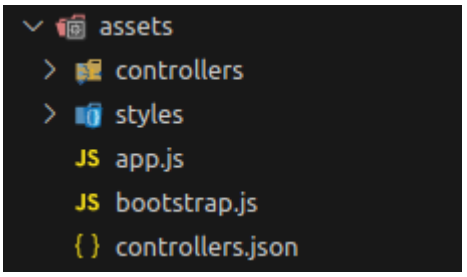
Src :
Les fichiers sources, partie centrale de l'application. Contient les fichiers PHP des contrôleurs, des entités, et des dépôts d'entités. C'est ici que se trouve tous les fichiers PHP que j'ai écrits.

Template :
Dossier où sont stockés les gabarits utilisés par le « moteur de template twig » intégré à Symfony. Ce sont les vues de notre application, toutes les pages qui vont être affichées à l'écran vont être ici.

Vendor :
Contient les « bundle » et dépendances externes créées avec composer et utilisés par l'application, comme par exemple le « bundle » de Symfony.

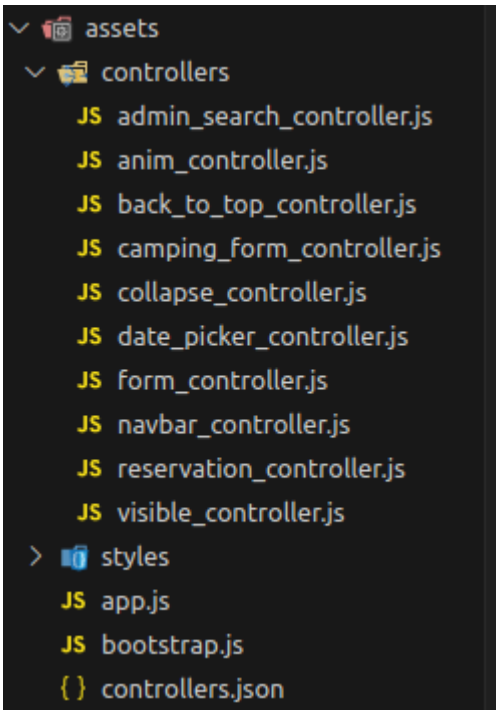
II - Développement de l'arborescence

1 - Dossier "Assets"



Racine :

- App.js , géré par webpack ce fichier répertorie les fichiers .js et .css en un seul point qui pourra être référencé dans les vues. Ici un seul de ces fichiers est nécessaire, toutes mes pages utilisant les mêmes feuilles de styles.
- Bootstrap.js, controllers.json , permet au bundle Stimulus de Symfony UX de fonctionner

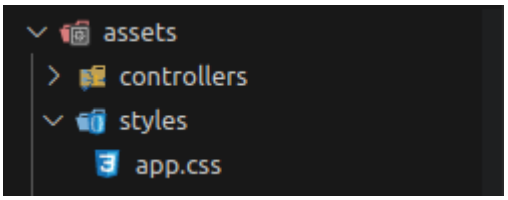


Controllers :

Contient les fichiers stockant les logiques javascript utilisées dans les vues. Chaque contrôleur se charge d'une fonctionnalité précise.

Ces fichiers important et étendent la classe 'Controller' de Stimulus

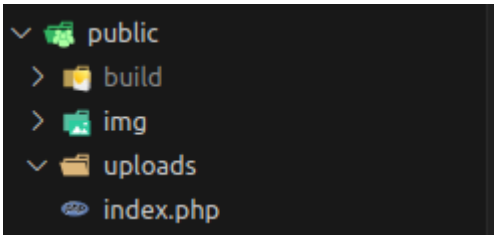
Ces contrôleurs sont ensuite rattachés aux éléments correspondants à l'aide d'attributs HTML.



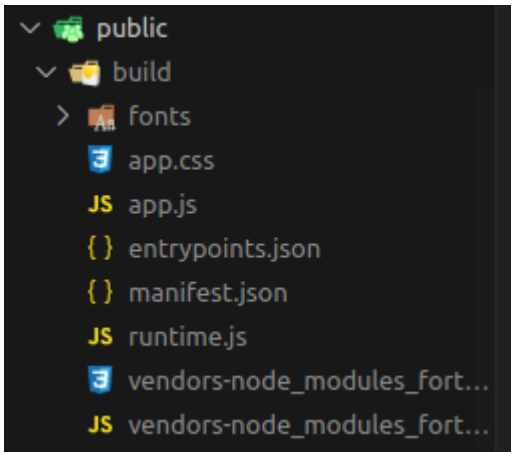
Styles :

Contient du css personnalisé, le css se base sur les couches tailwind @base, @components et @utilities

2 - Dossier "Public"

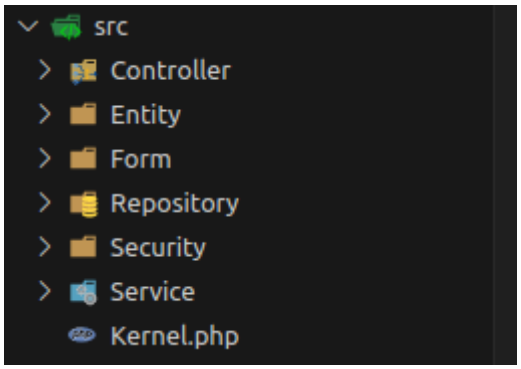


Index.php :
Contrôleur frontal de symfony.
Img :
Contient les images du site, utilisées dans les vues.
Uploads :
Contient les images téléchargées sur le serveur par les utilisateurs, le chemin de ses images est stocké en BDD.

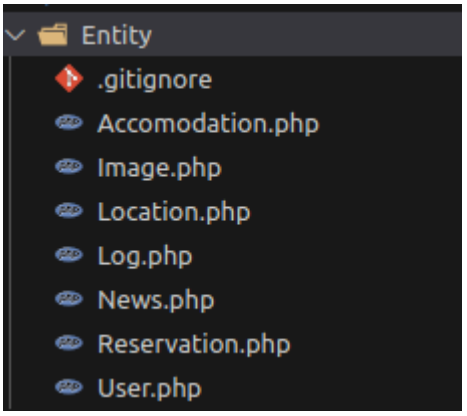


Build :
Contient les .css et .js compilés grâce à Webpack. Contient également des dépendances tels que les polices ou encore la librairie Fontawesome.

3 - Dossier "Src"

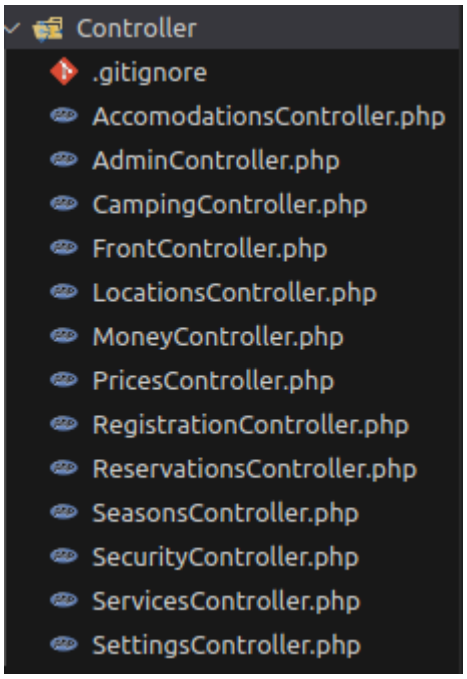


Regroupe les Controllers, les Entités, les Form, les Repository, la Security et les Services.



Entity :
Chaque fichier représente une classe possédant des propriétés ainsi que des méthodes.

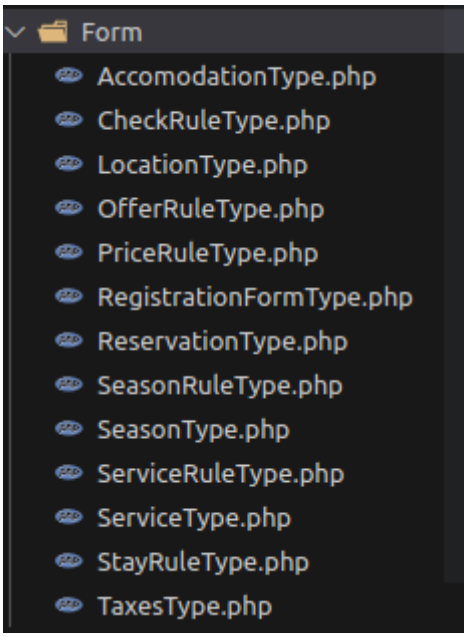
Ces classes sont fidèles aux tables de la base de données et ont été généré via la commande de symfony “php/bin console make:entity”



Controller :

Ces fichiers contrôlent le flux des données et mettent la vue à jour quand c'est nécessaire.
Chaque action du site passera par un de ces fichiers avant de se résoudre en succès ou échec.

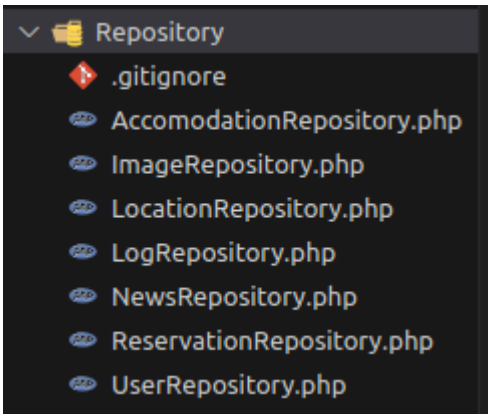
Chaque Controller se spécialise dans un domaine de l'application.



Form :

Chaque fichier représente une classe implémentant l'interface 'FormBuilderInterface' et possédant la méthode 'buildForm'.
Cette méthode pourra être appelée dans un **Controller** pour générer un formulaire et le traiter.
Ce formulaire pourra être envoyé à une **Template** afin d'être affiché à l'écran.

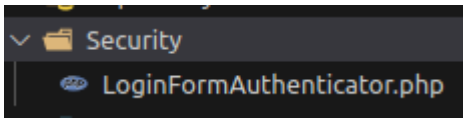
Certains formulaires prennent en paramètre une **Entité** à laquelle ils seront liés.
Ces formulaires ont été générés via la commande de symfony "php/bin console make:form"



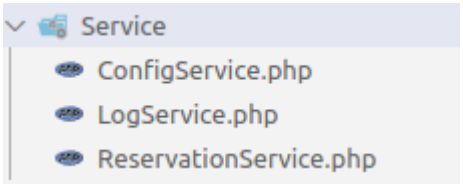
Repository :

Chaque fichier représente une classe étendant la classe 'ServiceEntityRepository' et étant lié à une classe **Entity**.

Permet grâce à **Doctrine** de faire un lien avec la base de données et de récupérer et manipuler des objets de classes correspondantes



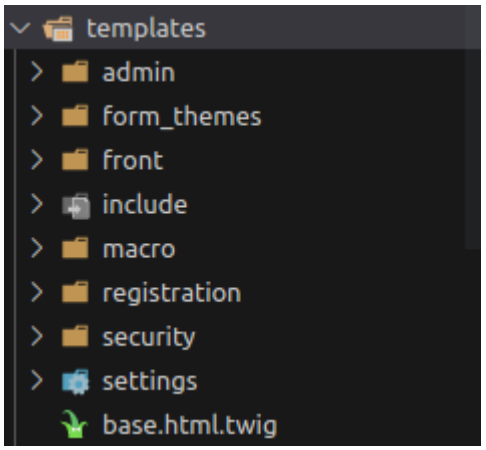
Security :
Contient l'authentificateur gérant la connexion.



Service :
Contient les services conçues dans le but de m'assister au développement.

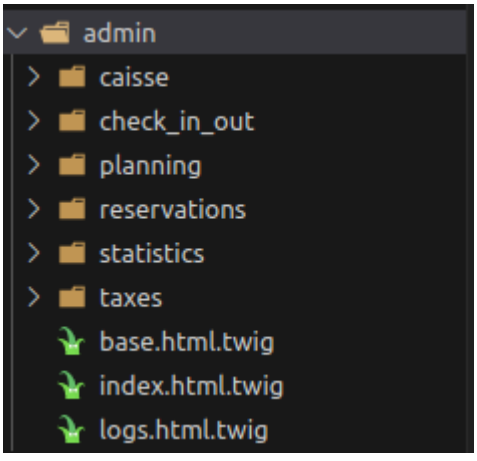
ConfigService me permet de communiquer avec le fichier 'config.json'
LogService me permet de générer les messages enregistrés dans le registre à chaque action administrateur
ReservationService gère le processus de recherche des réservations lors de la commande

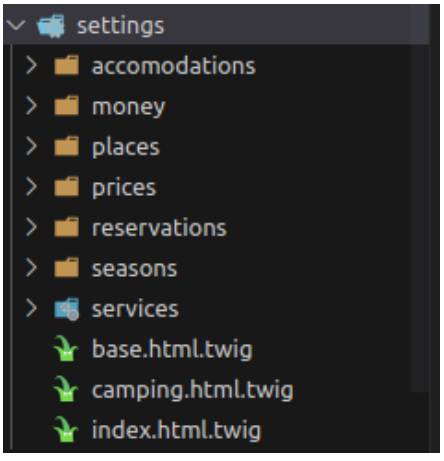
3 - Dossier "Template"

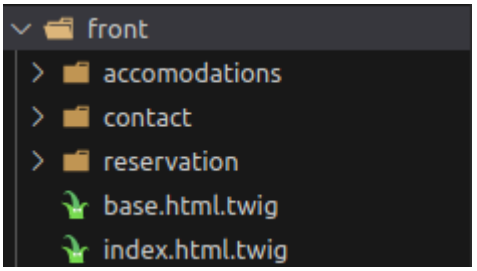


Racine :
Regroupe l'ensemble des dossiers contenant les vues du projet.
Les dossiers d'architecture de la même manière que le site, ainsi chaque dossier peut contenir plusieurs fichiers affichant les vues concernés, mais également d'autres sous dossiers.

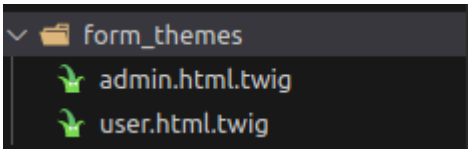
Le fichier base.html.twig contient la disposition de base étendue par les autres fichiers twig.



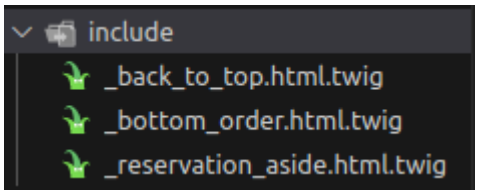




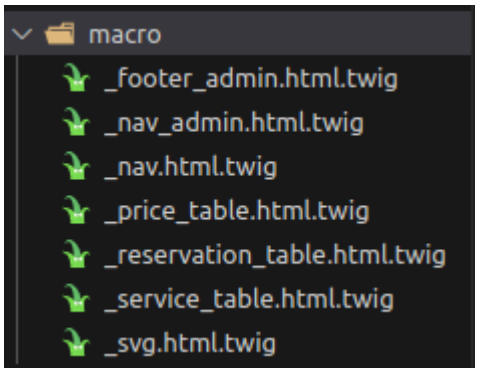
Admin : Contient les vues administrateurs réservés aux gestionnaires.
Settings : Contient les vues des paramètres du camping, réservés aux gestionnaires.
Front : Contient les vues du site ouvert aux publics



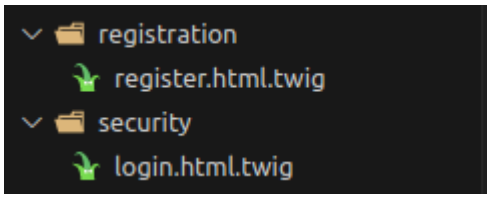
Form Themes :
Contient les thèmes des formulaires des vues twig. Ces thèmes sont appliqués par défaut dans le fichier twig.yaml



Include :
Contient les éléments html.twig utilisés à plusieurs endroits mais ne pouvant pas se trouver dans le fichier 'base.html.twig'



Macro :
Contient les macro twig permettant l'accès dans les vues à des variables contenant des éléments html.twig et auxquels on peut faire passer des paramètres. Exemple : le fichier 'svg.html.twig' contient tous les svg utilisés dans le site, il suffit d'importer le fichier et d'appeler la macro correspondante.



Registration & Security :
Contiennent les formulaires d'inscription et de connexion.

III- Modélisation de la base de données dans Symfony

1 – Création de la base de données

L'ORM (Object Relational Mapper) **Doctrine** permet de créer une correspondance entre un modèle objet et un modèle relationnel de base de données.
C'est donc lui qui gérera la base de données en la créant et la modifiant d'après un modèle d'objet.

La connexion a la base de données est récupérée depuis la variable d'environnement 'DATABASE_URL'

```
DATABASE_URL="mysql://nabil:root@localhost/camping?charset=utf8"
```

Si la base de données spécifiée n'existe pas, on peut la créer à l'aide de la commande "php bin/console doctrine:database:create"

```
doctrine:database:create                    Creates the configured database
```

Le CLI de symfony étant installé, je peux utiliser son raccourci à la place de 'php bin/console'. Je peux également utiliser les initiales comme alias.

```
uimm@HP-ProBook-450-G7:~/Bureau/symfony/camping$ symfony console d:d:c
Created database `camping2` for connection named default
uimm@HP-ProBook-450-G7:~/Bureau/symfony/camping$
```

2 – Création des Entity

La base de données créée, il faut maintenant générer les tables. La MCD a été conceptualisé sur le logiciel looping, mais c'est ici que l'on créera les entités et tables de la BDD.

La commande symfony "make:entity" permet de générer les entités

```
make:entity                 Creates or updates a Doctrine entity class, and optionally an API Platform resource
```

Nous allons l'utiliser pour créer l'entité **Accomodation**

```
imm@HP-ProBook-450-G7:~/Bureau/symfony/camping$ symfony console make:entity

Class name of the entity to create or update (e.g. GrumpyPopsicle):
> Accomodation
```

Deux fichiers ont été générés :

```
created: src/Entity/Accomodation.php
created: src/Repository/AccomodationRepository.php

Entity generated! Now let's add some fields!
You can always add more fields later manually or by re-running this command.
```

L'entité Accommodation a été générée avec comme seule propriété un id, et comme seule méthode un 'getId'

```php
#[ORM\Entity(repositoryClass: AccomodationRepository::class)]
47 references | 1 implementation
class Accomodation
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    1 reference
    private ?int $id = null;

    1 reference | 0 overrides
    public function getId(): ?int
    {
        return $this->id;
    }
}
```

Pour ajouter un champ voulu, on renseigne son nom puis on suit les étapes permettant de configurer la propriété.

```
New property name (press <return> to stop adding fields):
> name

Field type (enter ? to see all types) [string]:
> string

Field length [255]:
> 255

Can this field be null in the database (nullable) (yes/no) [no]:
> no

updated: src/Entity/Accomodation.php
```

```php
#[ORM\Column(length: 255)]

private ?string $name = null;
```

L'entité est mise à jour avec une nouvelle propriété, ainsi que deux nouvelles méthodes permettant d'attribuer et récupérer sa valeur. (des getters et setters)

On répète l'opération autant de fois que nécessaire jusqu'à avoir tous les champs de la table, tel que conçu sur la MCD.

```php
public function getName(): ?string
{
    return $this->name;
}

0 references | 0 overrides
public function setName(string $name):
{
    $this->name = $name;

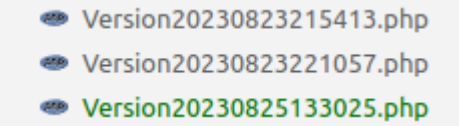
    return $this;
}
```

3 – Migration

Pour l'instant les modifications effectuées sont présentes dans nos fichiers sous forme d'un modèle d'objets Entity, chacune de ces Entity est censée représenter une table de notre BDD. Cependant pour l'instant notre BDD n'a aucune table.

Le rôle de doctrine est de lier ce modèle d'objet et la base de donnée, il va donc pouvoir générer un script SQL permettant de reproduire notre modèle d'objet dans une base de donnée à l'aide de la commande symfony 'make:migration'

```
make:migration                Creates a new migration based on database changes
```



Cette commande génèrera un fichier de migration possédant le script SQL correspondant aux modifications à apporter

```php
public function up(Schema $schema): void
{
    // this up() migration is auto-generated, please
    $this->addSql('CREATE TABLE accomodation (id INT
    $this->addSql('CREATE TABLE location (id INT AUTO
    $this->addSql('CREATE TABLE tag (id INT AUTO_INCR
    $this->addSql('CREATE TABLE messenger_messages (i
}

0 references | prototype
public function down(Schema $schema): void
{
    // this down() migration is auto-generated, pleas
    $this->addSql('DROP TABLE accomodation');
    $this->addSql('DROP TABLE location');
    $this->addSql('DROP TABLE tag');
    $this->addSql('DROP TABLE messenger_messages');
}
```

Ces fichiers de migrations servent aussi de versionning.
Une fonction contient le SQL permettant de mettre à jour la BDD,
tandis qu'une autre contient le SQL permet d'annuler ses modifications.

Cela permet de revenir en arrière à tout moment.

Pour executer le script SQL de la dernière version, on utilise la commande suivante :

```
doctrine:migrations:migrate   [migrate] Execute a migration to a specified version or the latest available version.
```

4 – Création des relations

Une fois nos entités définies, nous pouvons créer des relations entre elles.
Pour ça il faut modifier l'entité voulue avec la commande 'make:entity' et le nom de l'Entité, puis au moment d'attribuer la nouvelle variable on spécifie un des 4 types de relations

Ici on ajoute un champ **Location** à la table **Accomodation** avec comme type de relation 'OneToMany'

```
> uimm@HP-ProBook-450-G7:~/Bureau/symfony/camping$ symfony console make:entity Accomodation
Your entity already exists! So let's add some new fields!

New property name (press <return> to stop adding fields):
> location

Field type (enter ? to see all types) [string]:
> OneToMany

What class should this entity be related to?:
> Location

A new property will also be added to the Location class so that you can access and set the re
```

Cette relation signifie qu'une entité **Location** ne peut être lié qu'à une entité **Accommodation** tandis qu'une **Accommodation** peut être lié à plusieurs **Location**.

Si on est incertain du type de relation, la commande peut nous accompagner dans ce choix.

```
Add another property? Enter the property name (or press <return> to stop adding fielc
> tags

Field type (enter ? to see all types) [string]:
> relation

What class should this entity be related to?:
> Tag

What type of relationship is this?
------------ ---------------------------------------------------------------------
 Type         Description
------------ ---------------------------------------------------------------------
 ManyToOne    Each Accomodation relates to (has) one Tag.
              Each Tag can relate to (can have) many Accomodation objects.

 OneToMany    Each Accomodation can relate to (can have) many Tag objects.
              Each Tag relates to (has) one Accomodation.

 ManyToMany   Each Accomodation can relate to (can have) many Tag objects.
              Each Tag can also relate to (can also have) many Accomodation objects.

 OneToOne     Each Accomodation relates to (has) exactly one Tag.
              Each Tag also relates to (has) exactly one Accomodation.
------------ ---------------------------------------------------------------------
```

Ainsi les relations seront créées et les méthodes correspondantes générées. Une migration doit être générée et exécutée pour prendre les modifications en compte.

IV – Les outils développés

1 – Fonctionnalitées principales

1.1 - Règles de réservation

1.1.1 - Création de la vue

Pour accéder aux paramètre, un utilisateur connecté doit posséder le rôle ‘ROLE_ALLOWED_TO_CHANGE_SETTINGS’

```
- { path: ^/admin/settings, roles: ROLE_ALLOWED_TO_CHANGE_SETTINGS }
```

Ce rôle n’est pas possédé par les simples administrateurs :

```
role_hierarchy:
    ROLE_ADMIN:
    ROLE_SETTINGS: [ROLE_ADMIN, ROLE_ALLOWED_TO_CHANGE_SETTINGS]
    ROLE_LOGS: [ROLE_SETTINGS, ROLE_ALLOWED_TO_SEE_LOGS]
    ROLE_SUPER_ADMIN: [ROLE_LOGS, ROLE_ALLOWED_TO_CREATE_ADMINS]
```

Un lien sur la page d’accueil lui permettra d’y accéder, l’url est générée grâce à la fonction ‘path’ de twig, prenant en paramètre le nom de la route.

```
{% if admin %}
    <a class="flex gap-2 items-center" href="{{ path('admin_settings') }}">
        <p class="text-2xl">Paramètres</p>
```

Cette route est définie grâce à un attribut dans le contrôleur,

```
#[Route('/admin/settings', name: 'admin_settings')]
6 references | 0 overrides
public function index(): Response
{
    return $this->render('admin/settings/index.html.twig');
}
```

Cet attribut sera remarqué grâce à la configuration routes.yaml

config > routes.yaml > {} controllers

```
1  controllers:
2      resource:
3          path: ../src/Controller/
4          namespace: App\Controller
5      type: attribute
```

Le contrôleur utilise le ConfigService pour interagir avec le fichier de configuration et récupérer les règles de réservations et les envoyer sous forme de variables à twig.

```php
#[Route('/admin/settings/reservations', name: 'admin_settings_reservations')]
6 references | 0 overrides
public function settings(ConfigService $configService): Response
{
    $checkin = $configService->getReservationsRules('checkIn');
    $checkout = $configService->getReservationsRules('checkOut');
    $minstay = $configService->getReservationsRules('minStay');
    $maxstay = $configService->getReservationsRules('maxStay');
    $places = $configService->getReservationsRules('places');

    $reservations = $configService->getReservationsRules();

    return $this->render('admin/settings/reservations/index.html.twig', [
        'checkin' => $checkin,
        'checkout' => $checkout,
        'minstay' => $minstay,
        'maxstay' => $maxstay,
        'places' => $places,
        'reservations' => $reservations,
    ]);
}
```

Ces variables sont ensuite appelés dans le fichier twig pour être affichés à l'écran

```twig
<div class="container mx-auto mb-8">

    {{ macros.check_table(checkin,'checkIn') }}

    {{ macros.check_table(checkout,'checkOut') }}

    {{ macros.stay_table(minstay,'minStay') }}

    {{ macros.stay_table(maxstay,'maxStay') }}

</div>
```

Dans ce cas ci, j'utilise des macro, prenant en paramètre la règle de réservation

```twig
{% import "macro/_reservation_table.html.twig" as macros %}
```

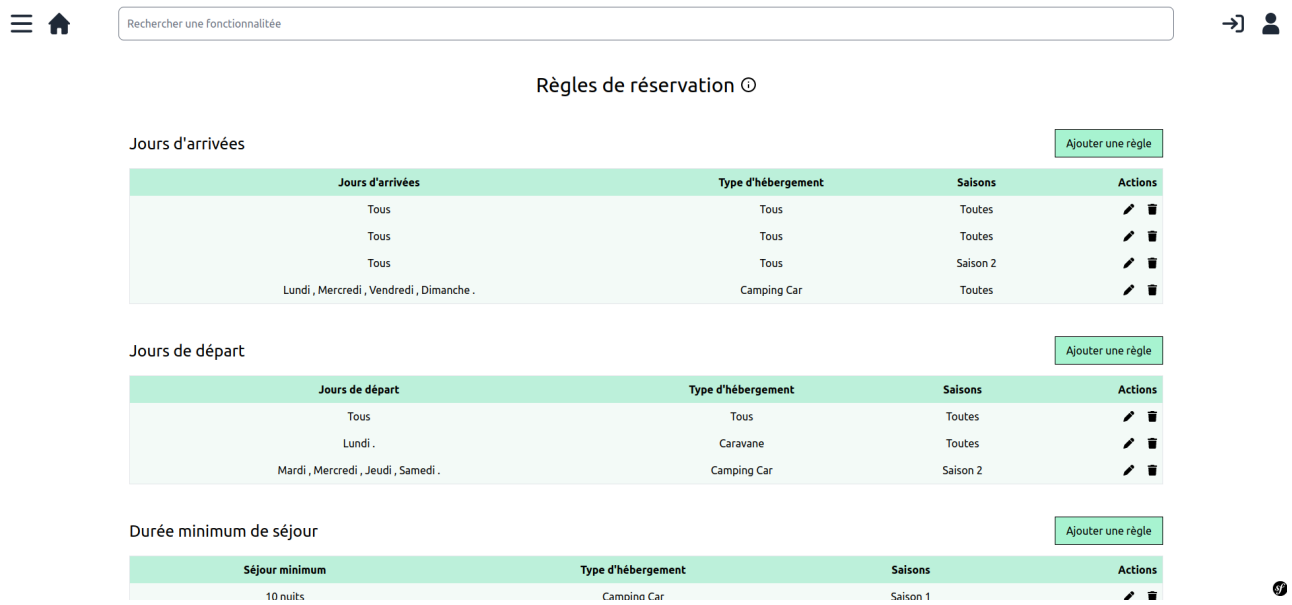
Ces macro permettent d'afficher plusieurs tables en limitant les répétitions de code et en facilitant les modifications et en améliorant la lisibilité du code.

```twig
{% macro check_table(check, type) %}…
{% endmacro %}

{% macro stay_table(stay, type) %}…
{% endmacro %}
```

1.1.2 - Interaction utilisateur

L'administrateur à donc accès à la liste des règles de réservations présentes dans le fichier de configuration. Il a la possibilité d'en ajouter, modifier et supprimer.



En cliquant sur "Ajouter une règle", l'utilisateur est à nouveau redirigé grâce à la fonction 'path' de twig, prenant cette fois un paramètre en option

```
<a href="{{ path('admin_settings_reservations_add', {'type':type}) }}">
```

Ce paramètre prenant le nom de 'type' est une 'wildcard' (un caractère de remplacement' dont la valeur sera stockée dans une variable du même nom.

```
#[Route("/reservations/{type}/add", na
6 references | 0 overrides
public function addRule(string $type,
```

/reservations/minStay/add

Un formulaire est généré en fonction du type récupéré, ces formulaires sont des classes stockés dans le dossier 'src/Form' et générée via SymfonyMakerBundle

```
$isCheck = str_contains(strtolower($type), 'check') ? true : false;
$isStay = str_contains(strtolower($type), 'stay') ? true : false;

if ($isCheck) $form = $this->createForm(CheckRuleType::class);
if ($isStay) $form = $this->createForm(StayRuleType::class);
```

1.1.3 - Création du formulaire

La class ‘StayRuleType’ se charge donc de construire le formulaire, la méthode add() permet d’ajouter un champ. Elle prend en paramètre le nom du champ, le type de donnée, et un tableau d’options.
Parmi les options importantes on peut noter ‘label’ permettant de définir le nom affiché et ‘constraints’ permettant d’ajouter des contraintes grâce au ‘Validator’ de symfony.

Si l’on prend le premier champ comme exempl:

- On récupère un input avec comme nom et id ‘stay_rule_amount’,
- Avec le field type ’IntegerType’, cet input devient un type ‘number’
- Avec l’option label, on nomme le label ‘montant’
- Avec les options de constraints ‘NotBlank’ et ‘Positive’ on demande à ce que la valeur soit non nulle et positive.

A noter que ces vérifications sont effectués côté serveur, les vérifications front natives à html5 sont également présente, tous les champs sont stipulés comme requis de base

```php
public function buildForm(FormBuilderInterface $builder, a
{
    $builder
        ->add('amount', IntegerType::class, [
            'label' => 'montant',
            'constraints' => [
                new NotBlank,
                new Positive
            ]
        ])
        ->add('places', ChoiceType::class, [
            'label' => 'emplacements',
            'choices' => $this->cs->getPlacesChoices(),
            'multiple' => true,
            'constraints' => [
                new NotBlank
            ]
        ])
        ->add('seasons', ChoiceType::class, [
            'label' => 'saisons',
            'choices' => $this->cs->getSeasonsChoices(),
            'multiple' => true,
            'constraints' => [
                new NotBlank
            ]
        ]);
}
```

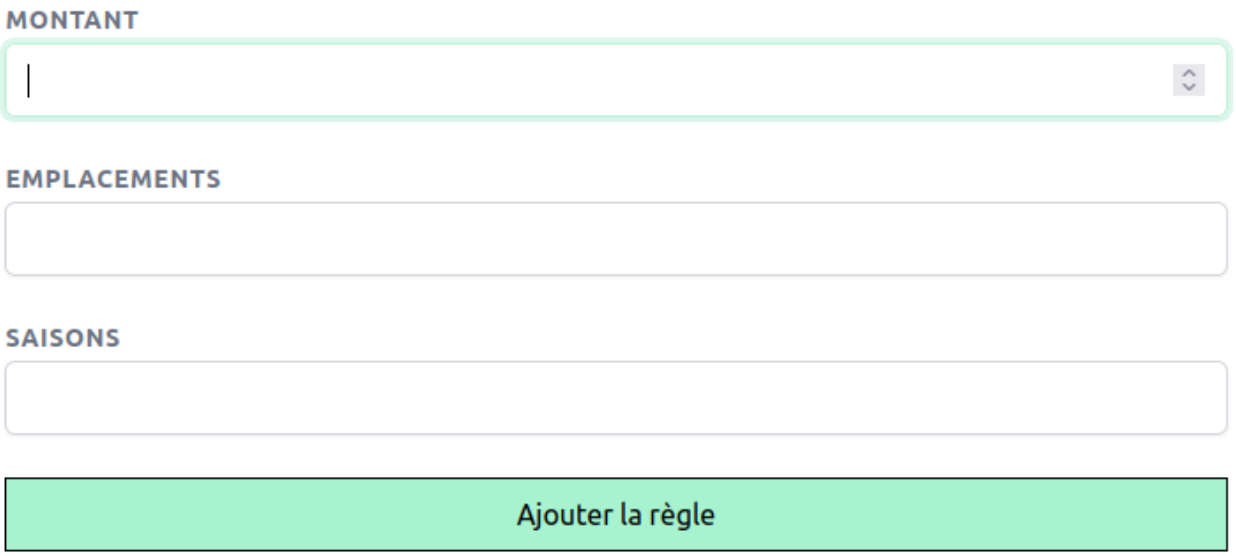
Le formulaire ainsi créé est passé à la template twig sous forme de variable

```php
return $this->render('admin/settings/reservations/add.html.twig', [
    'form' => $form,
    'type_name' => $type
]);
```

Il est ensuite affiché grâce aux fonctions ‘form’ de twig.

```twig
<div class="grid my-auto">
    {{ form_start(form) }}
    {{form_widget(form)}}
    <button type="submit" class="g-button w-full" >{{ add ? "Ajouter" : "Modifier" }}la règle</button>
    {{ form_end(form) }}
</div>
```

Voici le rendu obtenu dans ce cas ci.



Comme on peut le remarquer, le formulaire est déjà stylisé ! Cela est dû à une configuration permettant d’attribuer un thème de base aux formulaires :
Dans le fichier de configuration ‘twig.yaml’ on peut voir qu’un thème de formulaire a été défini.

config > packages > twig.yaml > { } twig

```yaml
twig:
  default_path: "%kernel.project_dir%/templates"
  form_themes: ["form_themes/admin.html.twig"]
```

Tout comme les extend des templates, utilisant les blocs d’un modèle en les écrasant si on les redéfinit.
Ici le thème de formulaire de tailwind est utilisé comme base, puis adapté pour convenir à l’identité du site.

templates > form_themes > admin.html.twig

```twig
{% use 'tailwind_2_layout.html.twig' %}
```

On peut voir des configurations générales pour tous les types d'input

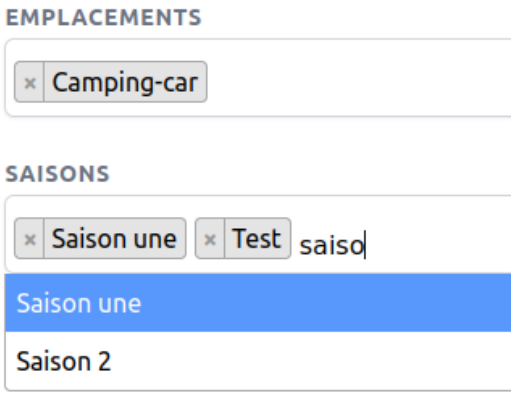
```
{# Custom de tous les labels #}
{%- block form_label -%}
    {%- set label_class = label_class|default('block text-gray-500 uppercas
    {{- parent() -}}
{%- endblock -%}

{# Custom de tous les inputs #}
{%- block widget_attributes -%}
    {%- set widget_class = widget_class|default('mt-1 block w-full rounded-
    {{- parent() -}}
{%- endblock -%}
```

Mais également d'autres plus précises pour certains types en particulier.

```
{# Modifie les input type="checkbox" #}
{%- block checkbox_widget -%}
    {%- set widget_class = widget_class|default('checked:bg-green-200 c
    {{- parent() -}}
{%- endblock -%}
```

Pour ce qui est des select à choix multiples, un bundle nommé 'Select2' se charge de les convertir en un composant bien plus simple et pratique d'utilisation.




```
$(document).ready(function() {
    $('select[multiple]').select2({
        closeOnSelect: false,
        language: 'fr',
        selectionCssClass: 'custom-select2'
    });
});
```

Ce simple script transforme tous les select avec l'attribut 'multiple'

En effet, le champ devient une barre de recherche activant un menu déroulant, ce menu comporte les différentes options qui peuvent être recherchés puis ajoutés par l'utilisateur.

En parlant des options, pour le cas d'une entité, spécifié le type 'EntityType' ainsi que la classe correspondante suffit à récupérer la liste des choix.
Cependant ici on parle de données présente dans le fichier de configuration 'config.json', ils ont été manuellement définis lors de la création du formulaire grâce à l'option 'choices' :

```
'choices' => $this->cs->getSeasonsChoices(),
```

```
public function getSeasonsChoices(): array
{
    $seasons = $this->getConfigByName('seasons');

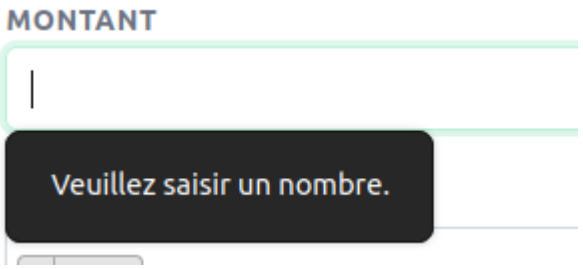
    $choices = ["Toutes" => null];
    foreach ($seasons as $season) {
        $choices[$season['name']] = $season['name'];
    }

    return $choices;
}
```

Cette méthode du ‘configService’ récupère donc la liste de toutes les saisons à mettre en tant qu’option pour les select multiple.

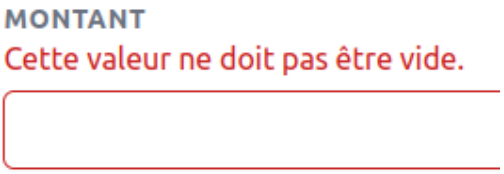
```php
public function getDaysChoices(): array
{
    return [
        "Tous" => null,
        "Lundi" => 1,
        "Mardi" => 2,
        "Mercredi" => 3,
        "Jeudi" => 4,
        "Vendredi" => 5,
        "Samedi" => 6,
        "Dimanche" => 7,
    ];
}
```

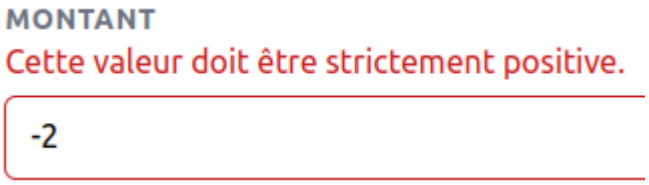
On notera que la clé est le nom affiché pour l’option, tandis que la valeur correspond à l’attribut HTML ‘value’.

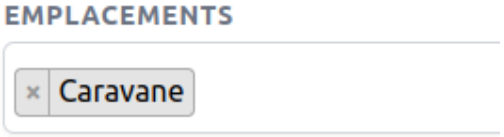


La vérification front-end HTML5 prend bien effet, cependant pour tester nous allons la désactiver et voir ce qu’il se passe :

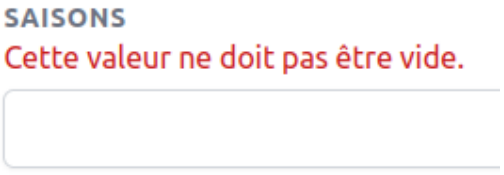
```
<button formnovalidate type="submit"
```







Des erreurs sont renvoyées par rapport aux contraintes définies plus haut, les champs concernés sont mis en avant.





Le contrôleur affichant le formulaire se charge également de le traiter, c’est lui qui définit si le formulaire est valide ou non, puis qui le renvoie ou procède au traitement.

Grâce au bundle ‘Turbo’ de Symfony UX, le site se ressent comme une ‘application à page unique’, tous les clicks et submissions de formulaires sont convertis en appel AJAX.

Grâce à ça, les données du formulaire sont traitées sans rechargement de page et ses erreurs sont affichées quasiment instantanément.

1.1.4 - Traitement du formulaire

Le formulaire implémente la class ‘formInterface’ possédant une méthode permettant d’inspecter une requête. On luit passe donc une instance de la classe ‘Requete’ de symfony, représentant la requête HTTP

On vérifie ensuite si le formulaire est soumis et bien valide. C’est ici que les contraintes sont vérifiées, si elles ne sont pas respectés on passe la boucle et retourne donc le formulaire, cette fois ci avec les erreurs en plus.

```
$form->handleRequest($request);

if ($form->isSubmitted() && $form->isValid()) {
```

Sinon, si le formulaire est valide alors on passe au traitement.

```
if ($form->isSubmitted() && $form->isValid()) {
    $lastId = $configService->getLastReservationRule($type);
```

Premièrement, le fichier de configuration ne possédant pas de système d’auto incrémentation pour ses ID, le nouvel ID doit être récupéré à la main

```
public function getLastReservationRule(string $type): int
{
    $lastId = $this->getReservationsRules($type);
    $lastId = end($lastId);

    return $lastId ? $lastId['id'] : 0;
}
```

Voici un exemple d’une règle de réservation dans le fichier de configuration:

```
    "minStay": [
        {
            "id": 1,
            "amount": 10,
            "places": [
                "Camping Car"
            ],
            "seasons": [
                "Saison 1"
            ]
        },
```

Il est important que le tableau à insérer respecte cette structure.
On lui attribue donc son id, puis on récupère les données du formulaire dans le même ordre que les champs

```
$rule = ["id" => $lastId + 1];
$rule += $form->getData();
```

On insère ensuite la règle de réservation dans le fichier à l'aide du 'configService', on utilise le 'type', ici 'minStay' afin de le placer au bon endroit.

```
$configService->addReservationRule($type, $rule);
```

Pour tenir compte des actions administrateur, j'ajoute un message flash en session (message s'auto-détruisant après utilisation) dans le but d'afficher une notification de succès.

Je fais également appel à mon 'LogService' pour insérer le message et son contexte en base de données.

```
$this->addFlash('success', 'Nouvel règle d\'emplacement ajouté avec succès!');
$message = "Une règle de réservation $type a été ajoutée";
$context = ['add', 'reservation'];
$logService->write($message, $context);
```

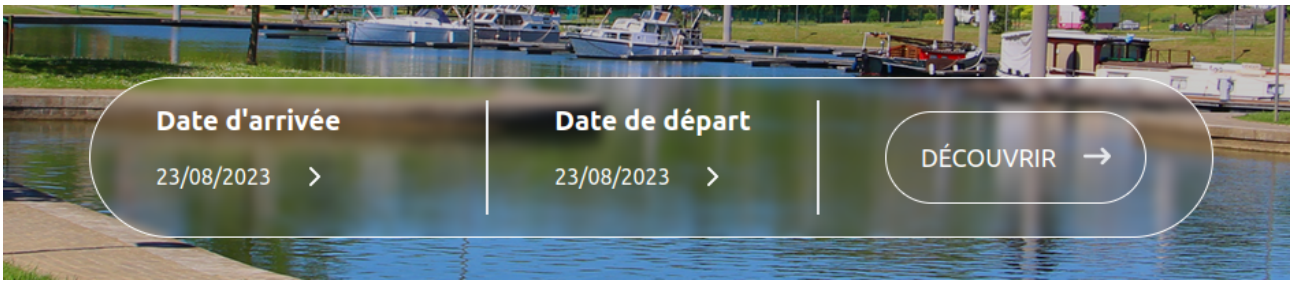
Enfin, le traitement correctement effectué, je redirige l'utilisateur vers la liste des règles de réservations à l'aide d'une méthode de l'abstractController symfony.

```
return $this->redirectToRoute('admin_settings_reservations');
```

1.2 - Réservation utilisateur

1.2.1 - Déclencheur utilisateur

Pour réserver l'utilisateur à plusieurs options,La principe est en tête de la page d'accueil, on y trouve un formulaire avec deux inputs pour les dates et un submit.



Ce formulaire est en réalité une section géré par un contrôleur stimulus 'date_picker'

```
<section id="reservation" {{ stimulus_controller('anim date_picker reservation') }} c
    <section id="start-date" class="relative pb-4 pr-12">
        <h2 class="font-bold text-xl mb-4">Date d'arrivée</h2>
        <input name="start" type class="w-[10ch] bg-transparent border-none outline-r
        <label for="start">
            <i class="ml-4 fa fa-chevron-right"></i>
        </label>
    </section>
    <section id="end-date" class="relative pb-4 px-12">
        <h2 class="font-bold text-xl mb-4">Date de départ</h2>
        <input name="end" type class="w-[10ch] bg-transparent border-none outline-nor
        <label for="end">
            <i class="ml-4 fa fa-chevron-right"></i>
        </label>
    </section>
    <section id="order" class="pl-12 py-2 ">
        <button {{ stimulus_action('reservation','open') }} class="group relative g-b
                    hover:border-main-500 hover:bg-main-500 rounded-full">Déc
            <i class="ml-4 mt-[1px] group-hover:ml-0.5 text-gray-200 group-hover:
        </button>
    </section>
</section>
```

Le raccourci twig 'stimulus_controller' permet d'attribuer les contrôleurs voulus à l'élément HTML. Dans le code il devient comme tel :

```
data-controller="anim date-picker reservation">
```

Le raccourci twig 'stimulus_controller' attribue lui une action à effectuer en réponse à un événement. L'événement de base pour les boutons étant le click.

Ce contrôleur importe la librairie DateRangePicker et a pour rôle d'en instancier un nouveau composant.

Lors de l'initialisation je définis des options telle que la langue (FR), la date minime (aujourd'hui) et des valeurs par défaut (Un départ dans une semaine et une durée de 7 jours)

> controllers > JS date_picker_controller.js > default

```
import { Controller } from "@hotwired/stimulus";
import DateRangePicker from "flowbite-datepicker/DateRangePicker";
import { locales } from "../../node_modules/flowbite-datepicker/js
import fr from "../../node_modules/flowbite-datepicker/js/i18n/loc

export default class extends Controller {
    connect() {
        this.setDates()
        this.initializeDateRangePicker();
        this.setDefaultValues();
    }

    setDates() {
        locales.fr = fr.fr;

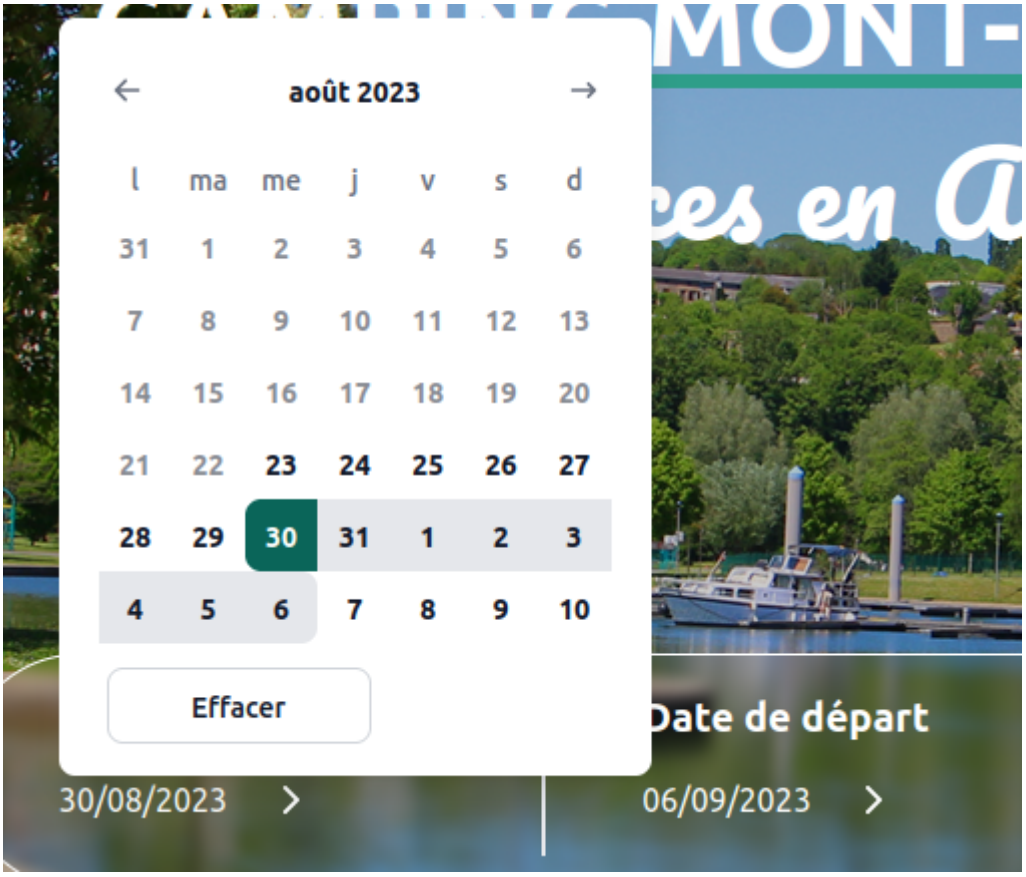
        let date = new Date();
        this.today = date.toLocaleDateString("fr-FR");
        this.nextWeek = new Date(
            date.getFullYear(),
            date.getMonth(),
            date.getDate() + 7
        ).toLocaleDateString("fr-FR");
    }

    initializeDateRangePicker() {
        const options = {
            language: "fr",
            weekStart: 1,
            clearBtn: true,
            minDate: this.today,
        };

        const d = new DateRangePicker(this.element, options);

        d.inputs.forEach((input) => {
            input.addEventListener("changeDate", (event) => {
                this.synchronizeValues();
            });
        });
    }
```

Le composant DateRangePicker se présente comme tel est permet à l'utilisateur de modifier les dates de début et de fin. Les options antérieures au jour actuel sont désactivées.



A chaque modification des dates par l'utilisateur, la méthode "synchronizeValues" est lancée. Cette méthode met à jour tous les sélecteurs de dates de la page avec les nouvelles valeurs (comme dit précédemment, il y en a plusieurs sur la page.)

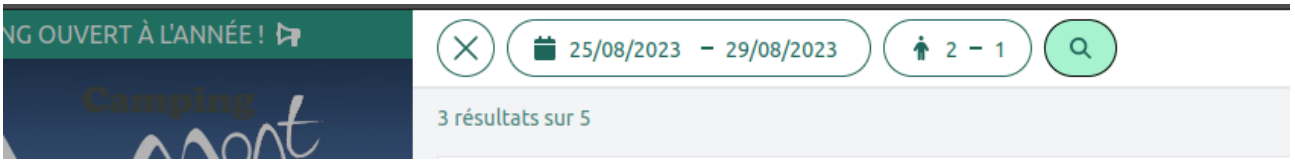
```
setDefaultValues() {
    this.start = this.element.querySelector('[name="start"]');
    this.end = this.element.querySelector('[name="end"]');

    this.start.value = this.nextWeek;
    this.end.value = this.nextNextWeek;
}

synchronizeValues() {
    let newStart = this.start.value;
    let newEnd = this.end.value;
    let startInputs = document.querySelectorAll('[name="start"]');
    let endInputs = document.querySelectorAll('[name="end"]');

    startInputs.forEach((input) => (input.value = newStart));
    endInputs.forEach((input) => (input.value = newEnd));
}
```

Lorsqu'il appuie sur le bouton, il déclenche la méthode 'open' ouvrant la barre latérale des réservations.



Cette barre latérale est liée au contrôleur 'reservation_controller.js'.

1.2.2 - Envoie de la requête au serveur

```
<section id="reservation-container" {{ stimulus_controller('reservation date_picker') }} class="
    <section id="top-bar" class="flex p-2 px-4 gap-4 items-center bg-white">
        <button id="close" {{ stimulus_action('reservation','close') }} class="p-1 aspect-square
            <i class="fa fa-xmark w-8 h-8"></i>
        </button>
```

La méthode principale de ce contrôleur est "find" qui se chargera de soumettre au serveur les informations de réservations du client, afin de récupérer sous forme de données les réservations disponibles correspondantes qu'il se charge d'afficher.

```
async find() {
    this.start = this.container.querySelector('input[name="start"]').value;
    this.end = this.container.querySelector('input[name="end"]').value;
    if (!this.start && !this.end) return;

    this.showLoader()
    let data = await this.fetchData(this.start, this.end);

    if(data.length > 0) this.insertData(data);
}
```

Dans le cas où les deux dates ne sont pas renseignées, aucune donnée n'est envoyée au serveur afin d'éviter du travail inutile. Sinon, un icône de chargement est affiché le temps de récupérer la réponse de la méthode fetchData.

```
async fetchData(start, end) {
    return fetch("/reservations/find", {
        method: "POST",
        headers: {
            "Content-Type": "application/json",
        },
        body: JSON.stringify({
            start: start,
            end: end,
            adults: this.adults,
            childs: this.childs,
        }),
    }).then((data) => data.json());
}
```

Cette méthode utilise l'api fetch qui est une interface native de javascript.
Elle prend en paramètre un url, la méthode ainsi que le contenu.

Ici les données sont envoyées en POST.

1.2.3 - Traitement des données sur serveur

Ces données sont interceptés dans le contrôleur ‘ReservationsController.php’
Le contrôleur utilise le système d'auto wiring de symfony, permettant de définir automatiquement les dépendances dans les contrôleurs.
C’est ainsi qu’il fait appel au composant ‘Request’ afin de récupérer le POST, avant de le décoder dans le but de traiter les données.

Est également appelé de la même façon le service ‘ReservationService’, service que j’ai créé dans le but de gérer le processus de réservation des clients.
Sa méthode ‘getReservationsByFormData’ est appelée dans le but de récupérer les réservations à proposer à l’utilisateur.

```php
#[Route('/reservations/find', name: 'reservations_find')]
6 references | 0 overrides
public function find(Request $request, ReservationService $rs, AccomodationRepository $ar): Response
{
    $postData = json_decode($request->getContent(), true);

    $reservations = $rs->getReservationsByFormData($postData);
    $message = $reservations;
    $message['count']['found'] = count($reservations['available']) + count($reservations['unavailable']);
    $message['count']['total'] = $ar->getTotalCount();

    return $this->json($message, 200);
}
```

Pour gérer ces dépendances, le service utilise lui son constructeur

```php
7 references | 0 implementations
class ReservationService
{
    2 references
    private AccomodationRepository $accomodationRepository;
    2 references
    private LocationRepository $locationRepository;
    2 references
    private ReservationRepository $reservationRepository;
    6 references
    private ConfigService $configService;

    2 references | 0 overrides
    public function __construct(AccomodationRepository $accomodatic
    {
        $this->accomodationRepository = $accomodationRepository;
        $this->locationRepository = $locationRepository;
        $this->reservationRepository = $reservationRepository;
        $this->configService = $configService;
    }
```

Cette méthode initie le processus de création des réservations en fonction des données reçues. Il récupère les emplacements disponibles sur la période demandée grâce à sa méthode ‘getAvailableLocationsByPeriod’

```php
1 reference | 0 overrides
public function getReservationsByFormData(array $data): array
{
    $locations = $this->getAvailableLocationsByPeriod($data['start'], $data['end']);
    $reservationsData = $this->getDisplayReservations($locations, $data);


    return $reservationsData;
}
```

Cette méthode utilise **AccomodationRepository** pour récupérer une liste d’entités des types d'hébergements mis en service. Avec la fonction array_map, on boucle sur tous les hébergements en leur appliquant la méthode ‘isAvailableDuringPeriod’ se chargeant de retourner un emplacement libre pour la période voulue.
L’array_filter lui sert à ne retourner que les emplacements non nuls

```php
1 reference | 0 overrides
public function getAvailableLocationsByPeriod(string|Date $start, string|Date $end): array
{
    $accomList = $this->accomodationRepository->findBy(['available' => true]);
    $locations = array_map(function ($accomodation) use ($start, $end) {
        return $this->isAvailableDuringPeriod($accomodation, $start, $end);
    }, $accomList);


    return array_filter($locations, function ($location) {
        return $location != false;
    });
}
```

La méthode mentionnée précédemment en utilise elle-même deux autres.
La première permet de récupérer les réservations du type d'hébergement durant une période.
La seconde utilise ces résultats pour récupérer les emplacements disponibles en fonction de l’une liste de réservations. Un 3ème paramètre optionnel permet de restreindre à un type d’hébergement.

```php
1 reference | 0 overrides
public function isAvailableDuringPeriod(Accomodation $accomodation, string|Date $start, string|Date $end): bool|Location
{
    // Récupère toutes les réservations de l'accomodation dont le séjour chevauche la période voulue
    $reservations = $this->reservationRepository->getReservationsByPeriod($start, $end, $accomodation);
    // Récupère tous les emplacements de l'accomodation qui ne sont pas réservés
    $locations = $this->locationRepository->getAccomodationLocationsByReservations($accomodation, $reservations, true);


    return empty($locations) ? false : $locations[array_rand($locations)];
}
```

La première des deux méthodes mentionnées précédemment se trouve dans ’ReservationRepository.php’ et utilise le constructeur de requête de doctrine pour récupérer les données de la BDD.
La requête récupère toutes les réservations dont la date de début de ou de fin se trouve parmi la plage horaire renseignée par le client.

```php
1 reference | 0 overrides
public function getReservationsByPeriod(string|DateTime $start, string|DateTime $end, Accomodation $accomodation = null,): array
{
    $start = DateTime::createFromFormat('d/m/Y', $start);
    $end = DateTime::createFromFormat('d/m/Y', $end);

    // Récupère toutes les réservations dont le séjour chevauche la période voulue
    $qb = $this->createQueryBuilder('r')
        ->join('App\Entity\Location', 'l', 'with', 'r.Location = l.id')
        ->andWhere('r.start BETWEEN :start AND :end')
        ->orWhere('r.end BETWEEN :start AND :end')
        ->setParameter('start', $start)
        ->setParameter('end', $end);
    // Récupère uniquement pour un hébérgement si spécifié
    if ($accomodation) {
        $qb->andWhere('l.accomodation = :accomodation')
            ->setParameter('accomodation', $accomodation->getId());
    }

    $reservations = $qb->getQuery()
        ->getResult();

    return $reservations;
}
```

La deuxième se trouve dans ‘LocationRepository.php’. Elle récupère tout d’abord les emplacements de la liste de réservation, puis effectue une requête en se basant sur celle-ci.
On récupère tous les emplacements disponibles d’un type d'hébergement dont l’identifiant se trouve ou non (spécifié par un paramètre booléen) dans la liste des emplacements récupérés.

Ainsi on peut retourner soit tous les emplacements utilisés, soit tous les emplacements libres d’une plage de dates.
Ici on retournera tous les emplacements libres avant d’en retourner un aléatoire dans la méthode parente.

```php
1 reference | 0 overrides
public function getAccomodationLocationsByReservations(Accomodation $accomodation, array $reservations, bool $reversed = false): array
{
    $condition = $reversed ? 'NOT IN' : 'IN';

    $locationIds = array_map(fn ($reservation) => $reservation->getLocation()->getId(), $reservations);

    $qb = $this->createQueryBuilder('l')
        ->andWhere("l.available = true")
        ->andWhere("l.accomodation = :accomodation")
        ->setParameter("accomodation", $accomodation);
    if (!empty($locationIds))
        $qb->andWhere("l.id $condition (:ids)")
        ->setParameter('ids', $locationIds);

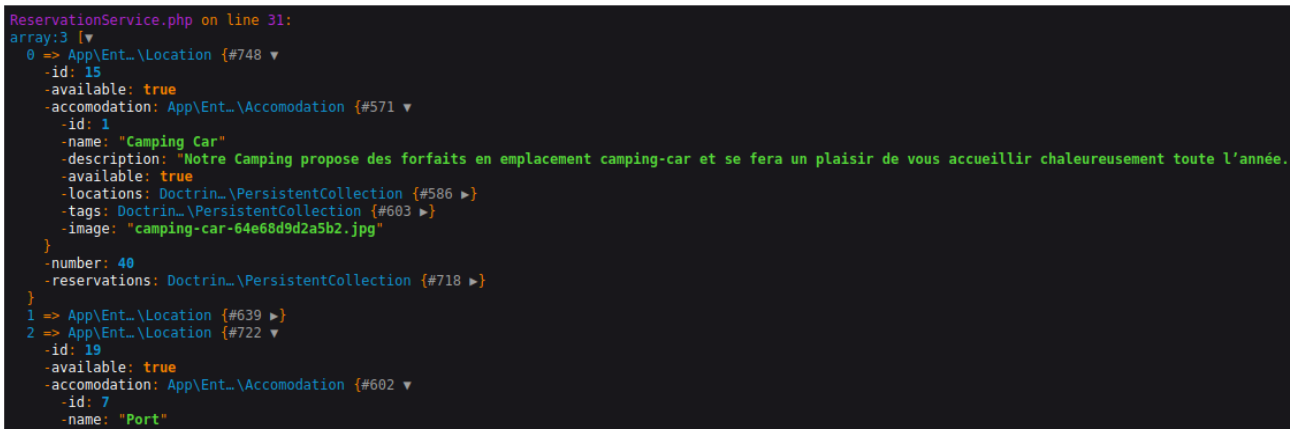

    return $qb->getQuery()
            ->getResult();
}
```

Nous nous trouvons donc actuellement à la moitié de la méthode initiale, avec une liste d'emplacements

```php
1 reference | 0 overrides
public function getReservationsByFormData(array $data): array
{
    $locations = $this->getAvailableLocationsByPeriod($data['start'], $data['end']);
    dd($locations);
    $reservationsData = $this->getDisplayReservations($locations, $data);


    return $reservationsData;
}
```

Voici le résultat si nous affichons les données actuelles :



Trois types d'emplacements différents se trouvent en BDD : 'Camping Car', 'Caravane' et 'Port'.
Cependant seulement deux emplacements ont été récupérés.
En effet, le Port ne possédant que trois emplacements, et ces derniers étant occupés pour la période du 23 au 30 août, aucun emplacement n'aura été retourné pour lui.

Les emplacements libres étant récupérés, il s'agit maintenant de vérifier grâce aux paramètres du camping, choisis par l'administrateur, si les règles de réservations sont respectées.
Certains emplacements ne pourront peut-être pas être proposés, dans ce cas il faudra retourner un message d'erreur au client pour lui expliquer quelle règle est enfreinte.

On utilisera la méthode 'getDisplayReservations' prenant en paramètres ces emplacements, ainsi que les données soumis par le client.

1.2.4 - Vérification des règles de réservations

Pour récupérer les réservations à afficher, on boucle sur les emplacements pour leur créer à chacun une réservation.
Pour communiquer avec le fichier de configuration, on utilise le 'configService'

```php
public function getDisplayReservations(array $locations, array $data): array
{
    $season = $this->configService->getSeasonByDate($data['start']);

    foreach ($locations as $location) {
        $reservation = new Reservation();

        $reservation->setLocation($location);
        $reservation->setStart($data['start']);
        $reservation->setEnd($data['end']);
        $reservation->setAdults($data['adults']);
        $reservation->setChilds($data['childs']);
```

On récupère la saison correspondant à la période grâce à la méthode 'getSeasonByDate', cette méthode boucle sur le fichier de configuration afin de trouver la saison correspondante aux dates

```php
public function getSeasonByDate(string $date): array
{
    $seasons = $this->getSeasons();

    foreach ($seasons as $seasonId => $seasonValue)
        foreach ($seasonValue['rules'] as $seasonRule)
            if ($this->isDateRangeMatching($seasonRule['start'], $seasonRule['end'], $date))
                $season = $seasons[$seasonId];

    return $season;
}
```

Après avoir récupéré la saison et créer un objet réservation,on les utilise dans une méthode du 'configService' se chargeant de vérifier toutes les règles de réservation, et de retourner une erreur si besoin en identifiant la cause.

```php
// Vérifie les règles de réservations
$error = $this->configService->checkReservationRules($reservation, $season);
```

La fonction commence par récupérer le jour de départ et d'arrivée sous forme d'un chiffre allant de 1 à 7 (Lundi à Dimanche).
Elle récupère ensuite depuis le fichier 'config.json' les jours d'arrivées et de départ autorisés pour cette réservation et par rapport à la saison.

Ces valeurs sont ensuite comparées afin de vérifier si elles sont respectées ou non, si ce n'est pas le cas, la fonction s'arrête et retourne le contexte de l'erreur.

L'opération est répétée avec le nombre de jours et les règles de durée minimum et maximum de séjour.

Si aucune erreur n'a été retournée, on renvoie un 'null'.

```php
1 reference | 0 overrides
public function checkReservationRules(Reservation $reservation, array $season)
{
    $startDay = $reservation->getStart()->format('N');
    $endDay = $reservation->getEnd()->format('N');
    $numberOfDays = $reservation->getStart()->diff($reservation->getEnd())->format('%a%');

    $checkInDays = $this->getReservationCheckIn($reservation, $season);

    if (!in_array($startDay, $checkInDays) && !in_array(null, $checkInDays))
        return $error = ['rule' => 'checkIn', 'value' => $checkInDays];

    $checkOutDays = $this->getReservationCheckOut($reservation, $season);

    if (!in_array($endDay, $checkOutDays) && !in_array(null, $checkOutDays))
        return $error = ['rule' => 'checkOut', 'value' => $checkOutDays];


    $minStay = $this->getReservationMinStay($reservation, $season);
    if ($minStay && $minStay > $numberOfDays)
        return $error = ['rule' => 'minStay', 'value' => $minStay];


    $maxStay = $this->getReservationMaxStay($reservation, $season);
    if ($maxStay && $maxStay < $numberOfDays)
        return $error = ['rule' => 'maxStay', 'value' => $maxStay];


    return null;
}
```

Ci-dessous, un visuel d'une des fonctions utilisée pour chaque règle.

Chacune de ces fonctions sont structurées de la même manière.

Certaines règles s'appliquent à tous les emplacements, tandis que d'autres sont spécifiques.
De même pour les saisons.

Il y a donc une hiérarchie permettant de retourner en priorité les règles les plus spécifiques.

```
1 reference | 0 overrides
public function getReservationCheckOut(Reservation $reservation, array $season)
{
    $rules = $this->getReservationsRules()['checkOut'];
    $accomodation = $reservation->getLocation()->getAccomodation();

    foreach ($rules as $checkOut) {
        if (in_array($accomodation->getName(), $checkOut['places'])) {

            if (in_array($season['name'], $checkOut['seasons']))
                $rule = $checkOut['days'];
            elseif (in_array(null, $checkOut['seasons']))
                $rule2 = $checkOut['days'];
        } elseif (in_array(null, $checkOut['places'])) {

            if (in_array($season['name'], $checkOut['seasons']))
                $rule3 = $checkOut['days'];
            elseif (in_array(null, $checkOut['seasons']))
                $rule4 = $checkOut['days'];
        }
    }

    return $rule ?? $rule2 ?? $rule3 ?? $rule4 ?? false;
}
```

Une fois les erreurs récupérées, on l'utilise pour catégoriser la réservation

```
// Vérifie les règles de réservations
$error = $this->configService->checkReservationRules($reservation, $season);

// Ajoute la card dans un array selon les erreurs
$displayReservations[$error ? 'unavailable' : 'available'][] = [
```

Cette array stockera toutes les cards de réservations qui devront être affichés comme disponibles et indisponibles

```
    // Ajoute la card dans un array selon les erreurs
    $displayReservations[$error ? 'unavailable' : 'available'][] = [
        'accomodation' => $location->getAccomodation()->getDisplayArray(),
        'location' => $location->getDisplayArray(),
        'price' => $this->getReservationPrices($reservation, $season),
        'error' => $error,
        'reservation' => [
            'start' => $data['start'],
            'end' => $data['end'],
            'adults' => $reservation->getAdults(),
            'childs' => $reservation->getChilds(),
            'stay' => $reservation->getStart()->diff($reservation->getEnd())->format('%a%'),
        ],
    ];
}
```

```
return $displayReservations ?? [];
```

L’array prend en valeurs plusieurs arrays, tel que accommodation pour les informations de l'hébergement :

```php
public function getDisplayArray(): array
{
    $tags = array_map(function($tag) { return $tag->getName() ;}, $this->getTags()->getValues());

    return [
        'id' => $this->id,
        'name' => $this->name,
        'description' => $this->description,
        'image' => $this->image,
        'tags' => $tags
    ];
}
```

Price pour les différents prix à afficher

```php
public function getReservationPrices(Reservation $reservation, array $season)
{
    $accomodation = $reservation->getLocation()->getAccomodation();

    $basePrice = $this->configService->getAccomodationPriceBySeason($accomodation->getId(), $season);
    $adultPrice = $this->configService->getAdultPriceBySeason($season);
    $childPrice = $this->configService->getChildPriceBySeason($season);

    $stay = $reservation->getStart()->diff($reservation->getEnd())->format('%a%');
    $totalAdult = $adultPrice * $reservation->getAdults();
    $totalChild = $childPrice * $reservation->getChilds();
    $totalPrice = ($basePrice + $totalAdult + $totalChild) * $stay;

    return [
        'accomodation' => $basePrice,
        'adult' => $adultPrice,
        'child' => $childPrice,
        'total' => $totalPrice,
    ];
}
```

ou encore l’erreur récupérée plus tôt, qui peut être nulle.
Le service de réservation aura enfin fini son traitement et envoie cet array comme réponse à la requête AJAX.

```php
public function getReservationsByFormData(array $data): array
{
    $locations = $this->getAvailableLocationsByPeriod($data['start'], $data['end']);
    $reservationsData = $this->getDisplayReservations($locations, $data);

    return $reservationsData;
}
```



Voici le format de la réponse JSON récupérée, visible dans la partie ‘Réseau’ de l’inspecteur

Si la réponse à au moins une réservation à afficher, elle appelle la méthode ‘insertData’ se chargeant d’insérer les données à l’endroit voulu.

```
this.showLoader();
let data = await this.fetchData(this.start, this.end);

if (data.count.found > 0) this.insertData(data);
```

La fonction affiche le nombre de résultats, puis boucle d’abord sur les réservations disponibles puis sur celles indisponibles.
Pour chacun d’entre elle, elle insère une carte dans le container en lui ajoutant une string à son innerHTML.
Ce string est généré par la méthode ‘getHTMLCard’ correspondante.

```
insertData(data) {
    this.cardsContainer.innerHTML = `<header class="text-sm opacity-80">${data.count.found} résultats sur ${data.count.total}</header>`;

    data.available.forEach((card) => this.cardsContainer.innerHTML += this.getAvailableHTMLCard(card) );

    data.unavailable.forEach((card) => this.cardsContainer.innerHTML +=this.getUnavailableHTMLCard(card) );
}
```

La fonction créer un string contenant chaque élément de l’array ‘tags’

```
getHTMLCard(card) {
    let tags = "";
    card.accomodation.tags.forEach((tag) => tags += `<div class='bg-main-100 rounded-full p-1 px-4 text-sm'>${tag}</div>` );

    return `
```

Elle retourne ensuite le composant à insérer

```
return `
    <article id="accom-${card.accomodation.id}" data-location-id="${card.location.id}" data-location-number="${card.location.number}" class="reservation-card available-reservation bg-white
        <section class="flex gap-4">
            <div class="image-container w-1/3 aspect-[16/12] border border-white outline-1 outline-gray-200">
                <img class="h-full" src="uploads/accomodations/${card.accomodation.image}">
            </div>
            <div class="description-container flex flex-col relative flex-1 mt-2">
                <div class="text-lg font-title">${card.accomodation.name}</div>
                <p class="text-sm opacity-50"><i class="fa-solid fa-users"></i> 2/6 personnes | <i class="fa-solid fa-bed"></i> 8/16 emplacements restants</p>
                <div class="flex flex-wrap gap-2 mt-2">${tags}</div>
                <div class="flex items-center self-end place-self-end gap-2 w-fit h-fit mt-auto">
                    <p class="text-xs opacity-80">prix pour ${card.reservation.stay} nuits,</p>
                    <button data-action="click->reservation#price" data-div="price-${card.accomodation.id}" class="text-xs opacity-80 underline hover:opacity-100">voir le détail du prix</butt
                    <button class='bg-main-400 text-white rounded-full py-2 px-4'>${card.price.total}€ <i class="fa fa-chevron-right"></i></button>
                </div>
            </div>
        </section>
        <div id="price-${card.accomodation.id}" class="hidden my-2 text-sm opacity-80">
            <span class="font-bold"> Prix de l'hébergement : ${card.price.accomodation}€ </span> <br>
            Du ${card.reservation.start} au ${card.reservation.end} (${card.reservation.stay} nuits x ${card.price.accomodation}€) : ${card.price.accomodation * card.reservation.stay}€ <br>
            <span class="font-bold"> Adultes supplémentaires (${card.reservation.adults}) : ${card.price.adult}€ </span> <br>
            Du ${card.reservation.start} au ${card.reservation.end} (${card.reservation.stay} nuits x ${card.reservation.adults} x ${card.price.adult}€) : ${card.price.adult * card.reservatio
            <span class="font-bold"> Enfants supplémentaires (${card.reservation.childs}) : ${card.price.child}€ </span> <br>
            Du ${card.reservation.start} au ${card.reservation.end} (${card.reservation.stay} nuits x ${card.reservation.childs} x ${card.price.child}€) : ${card.price.child *card.reservation
        </div>
    </article>
`;
```

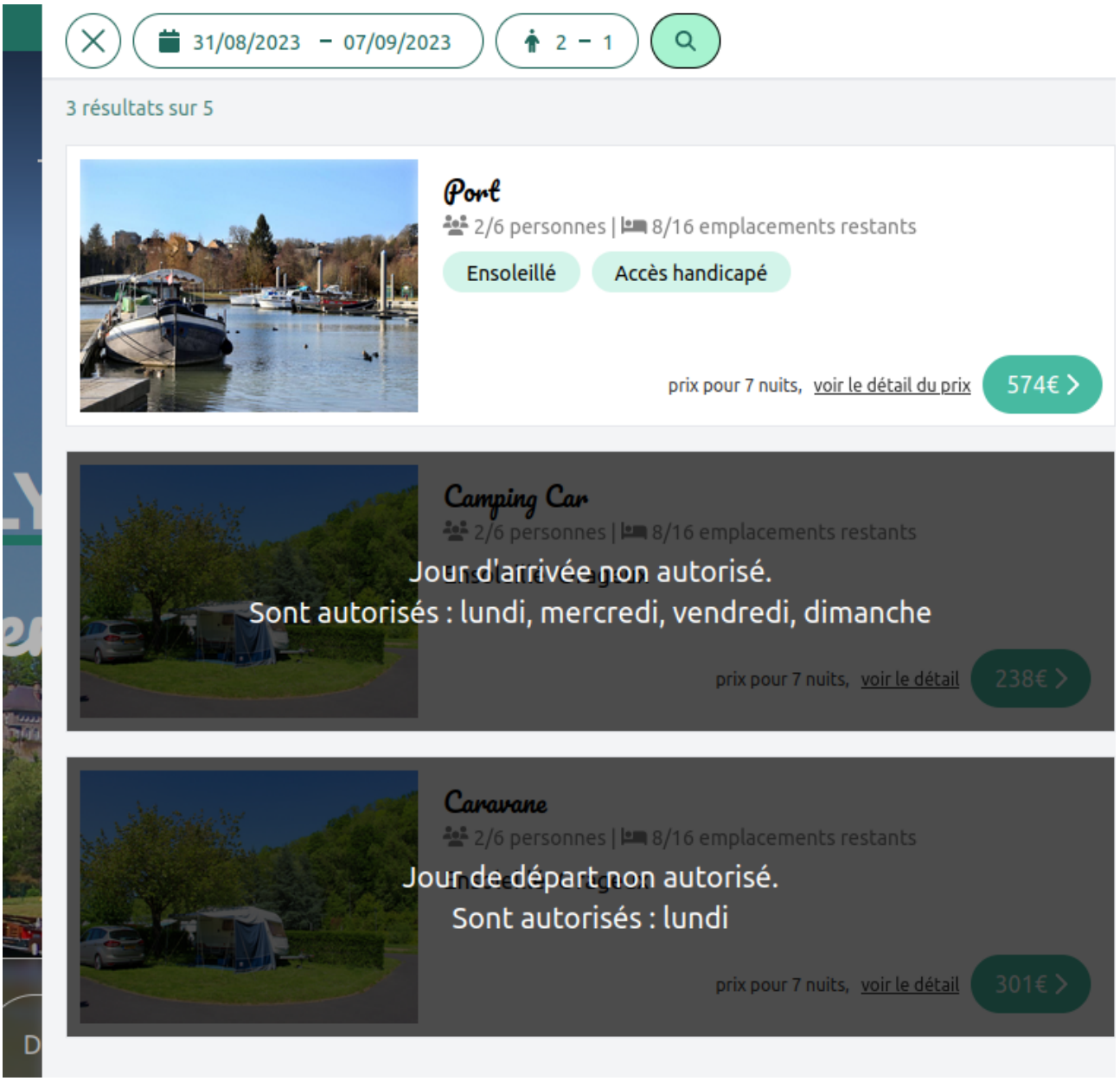
Pour les emplacements indisponibles, une méthode rend lisible le message d’erreur.

```
getErrorMessage(error) {
    let value = error.value;
    const days = ["lundi","mardi","mercredi","jeudi","vendredi","samedi","dimanche",];
    let messageDays = value.map((value) => days[value - 1]).join(", ");

    switch (error.rule) {
        case "checkIn":
            return `Jour d'arrivée non autorisé. <br> Sont autorisés : ${messageDays} `;
        case "checkOut":
            return `Jour de départ non autorisé. <br> Sont autorisés : ${messageDays} `;
        case "minStay":
            return `Le séjour doit durer au minimum ${value} nuits`;
        case "maxStay":
            return `Le séjour doit durer au minimum ${value} nuits`;
        case "full":
            return `Aucun emplacement disponible pour cette période`;
        default:
            return `Une règle de réservation n'a pas été respectée`;
    }
}
```

1.2.5 - Réponse renvoyée à l'utilisateur

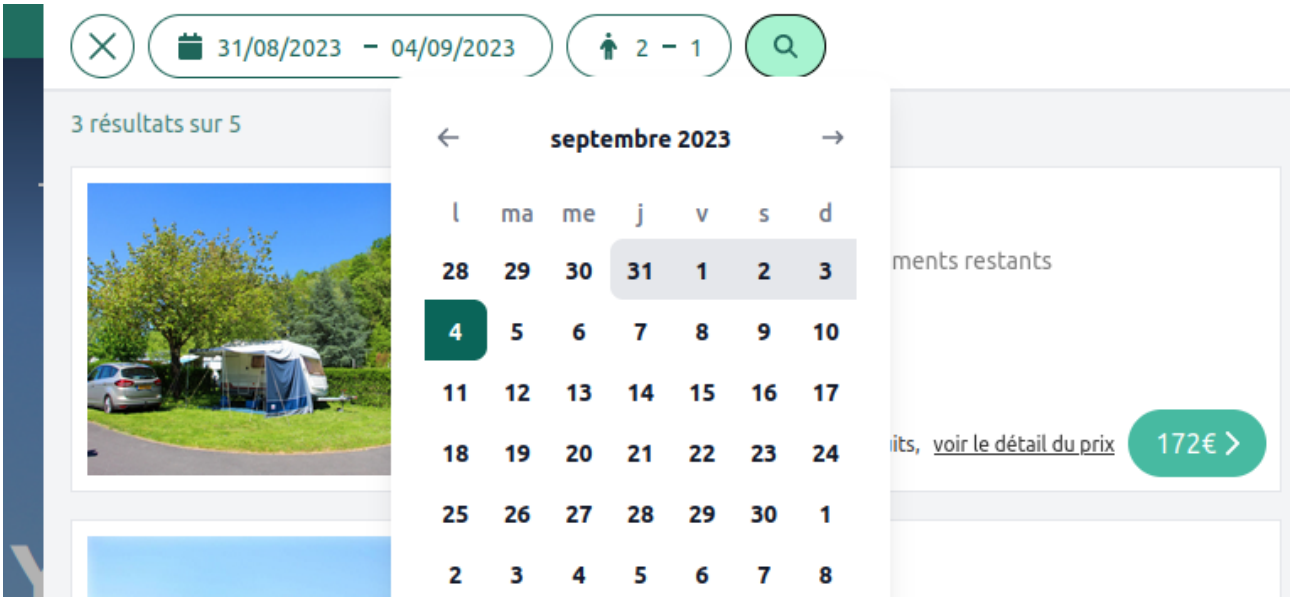
L'utilisateur a lancé la recherche, après un court chargement, les emplacements possibles lui sont montrés.



Il peut consulter le détail du prix en cliquant sur le bouton correspondant



Il voit également pour quelle raison certains emplacements sont indisponibles, pour réserver un emplacement caravane, il devra terminer son séjour un lundi :



La suite du traitement n'est pas encore prête mais voici la procédure prévue :

- L'utilisateur choisit de réserver un emplacement et passe à l'étape suivante
- Il choisit ou non des services supplémentaires (dont le prix s'ajoute au total)
- Il renseigne ses informations
- Un récapitulatif de la commande lui est montré avec le nouveaux prix et les taxes
- Il confirme la réservation et passe au paiement via Stripe
- La réservation est enregistrée en base de données et l'utilisateur reçoit un mail de confirmation.

1.3 - Registre administrateur

1.3.1 - Objectif et logique de la fonctionnalitée

Dans l'optique de pouvoir gérer au mieux le camping, un registre a été conçu. Grâce à lui on pourra retrouver quelles actions ont été effectuées à quel moment et par qui.
Un message doit donc être inscrit en base de données à chaque action entreprise par l'administrateur.
Le service 'LogService' sera chargé d'effectuer cette tâche

```
41 references | 0 implementations
class LogService
{
    3 references
    private EntityManagerInterface $entityManager;
    2 references
    private Security $security;
```

Sa méthode principale 'write' prend un message et un contexte en paramètre, et se charge d'insérer le message de registre correspondant.

```
27 references | 0 overrides
public function write(string $message, array $context):void
{
```

Une nouvelle instance de la classe 'Log' est créée, l'utilisateur connecté est récupéré puis ajouté à l'objet. Le message ainsi que son contexte sont également ajouté à l'objet.

```
$log = new Log;
$user = $this->security->getUser();
$log->setUser($user);
```

```
$log->setMessage($message);
$log->setContext($context);
```

Enfin, le gestionnaire d'entité de notre ORM se met à surveiller l'objet grâce à la méthode 'persist', puis l'ajoute en BDD lorsque la méthode 'flush' est utilisée.

```
$this->entityManager->persist($log);
$this->entityManager->flush();
```

```
#[ORM\PrePersist]
0 references | 0 overrides
public function prePersist(): void
{
    if ($this->getDate() === null)
        $this->setDate(new DateTime('now'));
}
```

La date est automatiquement ajoutée à l'objet avant qu'il soit intégré en BDD. Cela est possible grâce à l'attribut `#[ORM\HasLifecycleCallbacks]` sur la classe, et l'attribut `#[ORM\PrePersist]` sur la méthode

1.4 - Recherche administrateur

1.4.1 - Objectif et logique de la fonctionnalitée

Dans l'optique de faciliter le travail des administrateurs, une barre de recherche est toujours présente au haut de la page.

Le but de cette barre de recherche est de lister l'ensemble des fonctionnalités disponible à l'utilisateur, et lui permettre de rechercher ce qu'il souhaite afin de se rendre sur la page correspondante.



Toute la logique de la fonctionnalité se trouve dans un contrôleur stimulus, on attribue donc ce contrôleur à l'élément HTML concerné.

```
<section {{ stimulus_controller('admin_search') }} id="search-bar" class="relative grow md:mx-12 px-4 py-2">
```

```
<section {{ stimulus_controller('admin_sear
    <input {{ stimulus_action('admin_search
    <section {{ stimulus_target('admin_sear
</section>
```

Cet élément HTML est concerné de deux parties :
- L'input pour la barre de recherche
- Une section pour les résultats

1.4.2 - Fonctionnement du contrôleur

```
export default class extends Controller {
    static targets = ["results"];
```

Le contrôleur utilise le système de cibles de stimulus pour récupérer l'élément devant contenir les résultats.

L'élément est ciblé grâce au raccourci twig 'stimulus_target', équivalent de data-admin-search-target="results"

```
{{ stimulus_target('admin_search', 'results') }}
```

La méthode 'onSearchInput' est appelée à chaque Input de l'utilisateur

```
<input {{ stimulus_action('admin_search','onSearchInput','input')|s
```

La méthode 'search' est appelée lorsque l'input gagne le focus

```
)|stimulus_action('admin_search','search','focus')|
```

La méthode 'stop' est appelée lorsque l'input perds le focus

```
|stimulus_action('admin_search','stop','focusout')
```

La méthode ‘onSearchInput’ a pour seul objectif d’initier la méthode ‘search’ à chaque input à condition que le champ ne soit pas vide.

```
onSearchInput(e) {
    if (e.target.value.length > 0) this.search(e);
    else this.stop();
}
```

La méthode ‘search’ récupère les résultats à afficher, en comparant chaque élément d’un tableau défini à l’input utilisateur. Ces résultats sont une instance de la classe ‘Result’.

```
search(e) {
    this.resultsTarget.classList.remove("hidden");

    let results = values
        .filter((array) =>
            array[0].toLowerCase().includes(e.target.value.toLowerCase())
        )
        .map((array) => new Result(array[0], array[1]));

    this.insertResults(results);
}
```

Le tableau parmi lequel rechercher est défini dans le code tel quel :
Le nom affiché, la route correspondante

```
let values = [
    ["Consulter le registre", "admin/logs"],
    ["Consulter la Caisse", "admin/caisse"],
    ["Consulter le planning", "admin/planning"],
    ["Voir les arrivées et départs", "admin/checkInOut"],
    ["Accéder aux paramètres", "admin/settings"],
    ["Voir les Réservations", "admin/reservations"],
    ["Créer une Réservation", "admin/reservations/add"],
    ["Voir les Saisons", "admin/settings/seasons"],
    ["Ajouter une Saison", "admin/settings/seasons/add"],
    ["Voir les Services supplémentaires", "admin/settings/services"],
    ["Ajouter un Service supplémentaire", "admin/settings/services/add"],
    ["Voir les Hébergements", "admin/settings/accomodations"],
```

La fonction native ‘filter’ permet d’en créer une copie en appliquant une condition choisissant pour chaque valeur si elle est renvoyée ou non. Ici la condition est que la valeur doit comprendre le caractères entrés par l’utilisateur.

Ensuite la fonction native ‘map’ permet de boucler sur chaque valeur du tableau en appliquant une fonction, ici on transforme le tableau ‘values’ en une instance de la classe ‘Result’

La classe ‘Result’ a comme propriété le nom d’affichage et le chemin, comme pour l’array précédent. Cependant elle possède aussi une méthode permettant de générer un string HTML correspondant.

```
class Result {
    constructor(display, path) {
        this.display = display;
        this.path = path;
    }

    getHTML() {
        return `<article> <a href="/${this.path}"></a>${this.display}</article>`;
    }
}
```

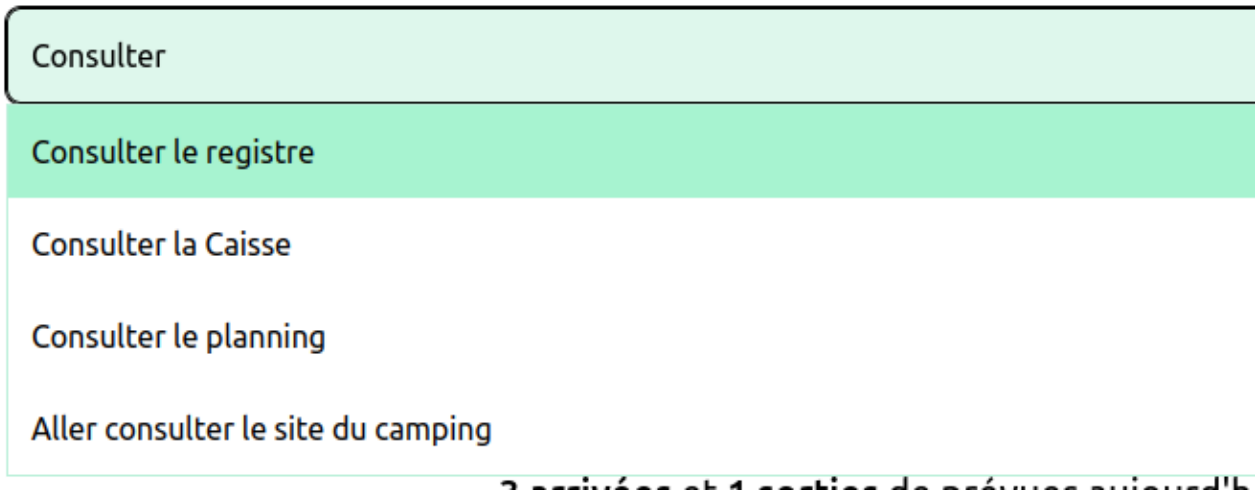
La section des résultats est vidée avant d’être remplie par les string HTML ainsi récupérées.

```
insertResults(results) {
    this.resultsTarget.innerHTML = "";

    results.forEach(
        (result) => (this.resultsTarget.innerHTML += result.getHTML())
    );
}
```

1.4.3 - Visuel utilisateur

Nous nous retrouvons avec ce résultat, cliquer sur une des valeur nous redirigera à la page voulue.



2 – Outils

2.1 - Visible_controller.js

2.1.1 - Objectif et logique du contrôleur

Dans le but dé gérer les animations de ma page d'accueil, j'ai créé un contrôleur stimulus permettant d'animer des éléments lorsqu'ils apparaissent à l'écran.

Il suffit d'attribuer le contrôleur 'visible' à l'élément qu'on souhaite animer, puis de lui attribuer 3 attributs de données :

- data-visible-classes : liste des classes à ajouter à l'élément une fois visible
- data-invisible-classes : liste des classes à ajouter à l'élément invisible
- data-transition-classes : classes de transitions voulus pour l'animation

```
</div>
<div class="flex-1 w-auto grid grid-cols-2 gap-4 p-4 mr-12">
    <figure>
        <img {{ stimulus_controller('visible') }} data-visible-classes="opacity-100" data-invisible-classes="opacity-0" data-transition-classes="duration-1000"
        class="h-full aspect-auto object-cover object-center" src="https://www.camping-mont-olympe.fr/wp-content/uploads/2019/04/IMG_5183-1024x683.jpg" alt="">
    </figure>
    <figure>
        <img {{ stimulus_controller('visible') }} data-visible-classes="opacity-100" data-invisible-classes="opacity-0" data-transition-classes="duration-1000"
        class="h-full  aspect-auto object-cover object-center" src="https://france3-regions.francetvinfo.fr/image/EYo8Qm63PdtDXY6ER82QBF7fXXY/800x450/filters:fc
    </figure>
</div>
```

Ici l'élément doit donc avoir une opacité de 0%, qui devra passer à 100% une fois l'élément visible à l'écran. Cette transition durera 1000ms soit 1s.

2.1.2 - Fonctionnement du contrôleur

Lors de l'initialisation du composant, on récupère un tableau des différentes classes

```
3   export default class extends Controller {
4       connect() {
5           let element = this.element;
6           let visibleClasses = element.dataset.visibleClasses.split(",");
7           let invisibleClasses = element.dataset.invisibleClasses.split(",");
8           let transitionClasses = element.dataset.transitionClasses.split(",");
```

Des fonctions permettent d'ajouter ou supprimer ces classes à un élément

```
function addClassesToElement(classes, element) {
    classes.forEach((className) => element.classList.add(className));
}

function removeClassesToElement(classes, element) {
    classes.forEach((className) => element.classList.remove(className));
}
```

On ajoute les classes de transition :

```
addClassesToElement(transitionClasses, element);
```

Une autre fonction se chargera d'assigner les classes visibles et invisibles en fonction de si l'élément est affiché à l'écran ou non

```
function applyClasses() {
    if (isVisible(element)) {
        addClassesToElement(visibleClasses, element);
        removeClassesToElement(invisibleClasses, element)
    } else {
        removeClassesToElement(visibleClasses, element);
        addClassesToElement(invisibleClasses, element);
    }
}
```

Un élément est considéré comme visible à l'écran lorsque sa bordure supérieure se trouve au moins à 100 pixels du bas de l'écran.

```
function isVisible(element) {
    let rect = element.getBoundingClientRect();

    return window.innerHeight - rect.top >= 100;
}
```

On appelle donc la fonction une fois à l'apparition du composant, puis à chaque scroll de l'utilisateur. Et voila, lorsqu'on scroll et que l'élément apparaît au bas de l'écran, l'animation voulue s'effectuera !

```
applyClasses();

window.addEventListener("scroll", applyClasses);
```

2.1.3 - Visuel utilisateur

Exemple d'une animation en cours avec 4 cards au délai différent.




```
data-invisible-classes="opacity-0,translate-y-1/2" data-transition-classes="duration-700,delay-[100ms]"

data-invisible-classes="opacity-0,translate-y-1/2" data-transition-classes="duration-700,delay-[200ms]"

data-invisible-classes="opacity-0,translate-y-1/2" data-transition-classes="duration-700,delay-[300ms]"

data-invisible-classes="opacity-0,translate-y-1/2" data-transition-classes="duration-700,delay-[400ms]"
```

1.2 - Macro SVG

Le site utilise plusieurs SVG en tant qu'icônes, tous ces SVG sont répertoriés en tant que macro dans un fichier twig. Ces macros prennent en paramètre deux string.

emplates > macro > _svg.html.twig

```
 1 > {% macro seasons(classes = "w-36 h-36") %}…
28   {% endmacro %}
29
30 > {% macro camping(classes = "w-36 h-36") %}…
46   {% endmacro %}
47
48 > {% macro prices(classes = "w-36 h-36") %}…
59   {% endmacro %}
60
61 > {% macro places(classes = "w-36 h-36") %}…
75   {% endmacro %}
76
77 > {% macro services(classes = "w-36 h-36", color = "black") %}…
92   {% endmacro %}
93
94 > {% macro rules(classes = "w-36 h-36", color = "#000000") %}…
102  {% endmacro %}
103
104 > {% macro map(classes = "w-36 h-36", color = "#000000") %}…
114  {% endmacro %}
115
116 > {% macro logs(classes = "w-36 h-36", clore = "#000000") %}…
125  {% endmacro %}
```

Un pour les classes, définissant notamment la taille. Et un définissant la couleur.

```
{% macro services(classes = "w-36 h-36"   fill={{ color }}>
    <svg class={{ classes }} viewbox="0
```

```
    {{ svg.map('w-36 aspect-square','#2e9e89') }}
</div>
<div class="□bg-white border-2 ■border-main-500 text
ticle>

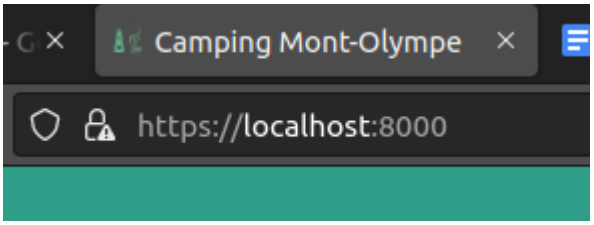
icle class="overflow-hidden group cursor-pointer">
<div class="relative rounded-full border-dotted border
    {{ svg.logs('w-36 aspect-square','#2e9e89') }}
</div>
<div class="□bg-white border-2 ■border-main-500 text
ticle>

icle class="overflow-hidden group cursor-pointer">
<div class="relative rounded-full border-dotted border
    {{ svg.services('w-36 aspect-square','#2e9e89') }}
</div>
```

Ces macros permettent de gagner grandement en lisibilité et de garder une cohérence dans tout le site.

Il sert aussi de répertoire pour ne pas avoir à rechercher les svg pour les réutiliser.

3 – La page "accueil" affichée dans le navigateur



4 – Connexion de l'utilisateur

Le bundle ‘security’ de symfony permet de faciliter la mise en place d’une inscription et connexion utilisateur

Ce bundle créer un fichier security.yaml qui nous permet de configurer un Hasher de password, le firewall qui prend en charge la connexion de l’utilisateur ainsi que l’autorisation pour l’utilisateur d’accéder à une page et l’access_control permettant la sécurisation des routes.

Pour créer un utilisateur on utilise la commande symfony ‘make:user’ :

```
> symuimm@HP-ProBook-450-G7:~/Bureau/symfony/camping$ symfony console make:user

The name of the security user class (e.g. User) [User]:
> User

Do you want to store user data in the database (via Doctrine)? (yes/no) [yes]:
> yes

Enter a property name that will be the unique "display" name for the user (e.g. email, u
> email

Will this app need to hash/check user passwords? Choose No if passwords are not needed o

Does this app need to hash/check user passwords? (yes/no) [yes]:
> yes

created: src/Entity/User.php
created: src/Repository/UserRepository.php
updated: src/Entity/User.php
updated: config/packages/security.yaml

 Success!

Next Steps:
  - Review your new App\Entity\User class.
  - Use make:entity to add more fields to your User entity and then run make:migration.
  - Create a way to authenticate! See https://symfony.com/doc/current/security.html
```

L’entité User ainsi générée est différente des autres étant donné qu’elle implémente deux interfaces permettant de gérer la connexion.

```php
#[ORM\Entity(repositoryClass: UserRepository::class)]
40 references | 1 implementation
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    #[ORM\Id]
```

Le registrationController se charge d’inscrire un utilisateur, le mot de passe est haché par le ‘PasswordHasherInterface’

```php
#[Route('/register', name: 'app_register')]
6 references | 0 overrides
public function register(Request $request, UserPasswordHasherInterface
{
    $user = new User();
    $form = $this->createForm(RegistrationFormType::class, $user);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        // encode the plain password
        $user->setPassword(
            $userPasswordHasher->hashPassword(
                $user,
                $form->get('plainPassword')->getData()
            )
        );
        $user->setRoles(['ROLE_ADMIN']);

        $entityManager->persist($user);
        $entityManager->flush();

        $message = "Un utilisateur " . $user->getFirstName() .  " " .$u
        $context = ['add','user'];
        $logService->write($message,$context);

        return $this->redirectToRoute('admin');
    }

    return $this->render('registration/register.html.twig', [
        'form' => $form
    ]);
}
```

C'est dans le fichier "security.yaml" que se réalise les paramètres de sécurité. Nous devons renseigner les encoders pour permettre l'encodage du mot de passe, les providers pour indiquer où aller chercher les informations (User) et lesquelles sont utilisées pour l'authentification (ici : Email) et les firewalls pour contrôler les routes de connexion.

```yaml
security:
    # https://symfony.com/doc/current/security.html#registering-the-user-hashing-passwords
    password_hashers:
        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: "auto"
    # https://symfony.com/doc/current/security.html#loading-the-user-the-user-provider
    providers:
        # used to reload user from session & other features (e.g. switch_user)
        app_user_provider:
            entity:
                class: App\Entity\User
                property: email
        # used to reload user from session & other features (e.g. switch_user)
    firewalls:
        dev:
            pattern: ^/(_(profiler|wdt)|css|images|js)/
            security: false
        main:
            lazy: true
            provider: app_user_provider
            custom_authenticator: App\Security\LoginFormAuthenticator
            logout:
                path: logout
                # where to redirect after logout
                # target: app_any_route

            # activate different ways to authenticate
            # https://symfony.com/doc/current/security.html#the-firewall

            # https://symfony.com/doc/current/security/impersonating_user.html
            # switch_user: true
    role_hierarchy:
        ROLE_ADMIN:
        ROLE_SETTINGS: [ROLE_ADMIN, ROLE_ALLOWED_TO_CHANGE_SETTINGS]
        ROLE_LOGS: [ROLE_SETTINGS, ROLE_ALLOWED_TO_SEE_LOGS]
        ROLE_SUPER_ADMIN: [ROLE_LOGS, ROLE_ALLOWED_TO_CREATE_ADMINS]

    access_control:
        - { path: ^/admin/users, roles: ROLE_ALLOWED_TO_CREATE_ADMINS }
        - { path: ^/admin/logs, roles: ROLE_ALLOWED_TO_SEE_LOGS }
        - { path: ^/admin/settings, roles: ROLE_ALLOWED_TO_CHANGE_SETTINGS }
        - { path: ^/admin, roles: ROLE_ADMIN }
```

5 – Sécurité

Doctrine utilise des requêtes paramétrées pour prévenir les injections SQL.
Cependant des types de données non voulus peuvent être enregistrés en BDD, c'est pourquoi des contraintes sont utilisées dans les formulaires.

Voici par exemple le traitement d'une image :
Le champ de type fichier n'autorise que les fichiers jpg, jpeg et png de taille inférieur à 8 mégabytes

```
->add('image', FileType::class, [
    'constraints' => [
        new File([
            'maxSize' => '8M',
            'mimeTypes' => [
                'image/jpg',
                'image/jpeg',
                'image/png',
            ],
            'mimeTypesMessage' => 'Seuls les images sont autorisés (png,jpg,jpeg)',
        ])
    ],
])
```

Le fichier est réceptionné dans le contrôleur et stocké à l'aide d'un nom temporaire, un nouveau nom lui est attribué avant qu'il soit déplacé sur le serveur dans le dossier configuré. Le nouveau nom est gardé en bdd afin de pouvoir récupérer le fichier.

```
$accomodation = $form->getData();

$imageFile = $form->get('image')->getData();

$originalFilename = pathinfo($imageFile->getClientOriginalName(), PATHINFO_FILENAME);
$safeFilename = $slugger->slug($originalFilename);
$newFilename = $safeFilename . '-' . uniqid() . '.' . $imageFile->guessExtension();

try {
    $imageFile->move(
        $this->getParameter('accomodations_directory'),
        $newFilename
    );
} catch (FileException $e) {
}

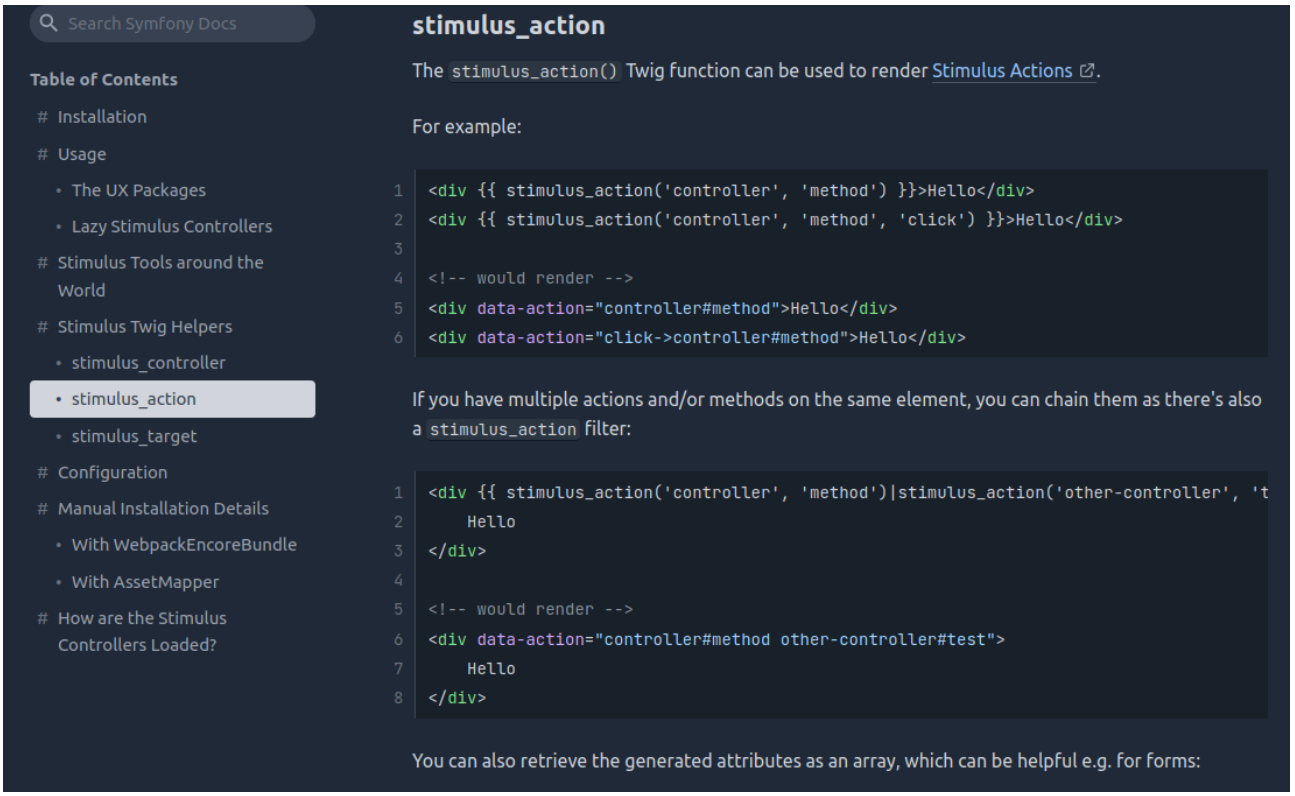
$accomodation->setImage($newFilename);
```

Symfony génère également les tokens CSRF

Exemple de veille technologique

Extrait de la documentation Symfony :

Comment attribuer plusieurs actions d'un contrôleur stimulus avec twig ?



Search Symfony Docs

Table of Contents

- # Installation
- # Usage
  - The UX Packages
  - Lazy Stimulus Controllers
- # Stimulus Tools around the World
- # Stimulus Twig Helpers
  - stimulus_controller
  - stimulus_action
  - stimulus_target
- # Configuration
- # Manual Installation Details
  - With WebpackEncoreBundle
  - With AssetMapper
- # How are the Stimulus Controllers Loaded?

stimulus_action

The `stimulus_action()` Twig function can be used to render Stimulus Actions.

For example:

```
<div {{ stimulus_action('controller', 'method') }}>Hello</div>
<div {{ stimulus_action('controller', 'method', 'click') }}>Hello</div>

<!-- would render -->
<div data-action="controller#method">Hello</div>
<div data-action="click->controller#method">Hello</div>
```

If you have multiple actions and/or methods on the same element, you can chain them as there's also a `stimulus_action` filter:

```
<div {{ stimulus_action('controller', 'method')|stimulus_action('other-controller', 't
    Hello
</div>

<!-- would render -->
<div data-action="controller#method other-controller#test">
    Hello
</div>
```

You can also retrieve the generated attributes as an array, which can be helpful e.g. for forms:

Utilisation dans le projet :

```
</section>
<section {{ stimulus_controller('admin_search') }} id="search-bar" class="relative grow md:mx-12 px-4 py-2">
    <input {{ stimulus_action('admin_search','onSearchInput','input')|stimulus_action('admin_search','search',
    <section {{ stimulus_target('admin_search', 'results') }} id="search-results" class="hidden absolute left-
</section>
```

Extrait de StackOverflow :

Comment définir des chemins d'accès aux dossiers d'un projet symfony ?

10

`$this->get('templating.helper.assets')` asset helper is about **HTTP URI**, not directory path. However, you can easily get the right absolute directory web path like this:

```
$appPath = $this->container->getParameter('kernel.root_dir');
$webPath = realpath($appPath . '/../web');
```

`$webPath` should return something like this:

```
/home/martins/www/symfony-project/web
```

Utilisation dans le projet :

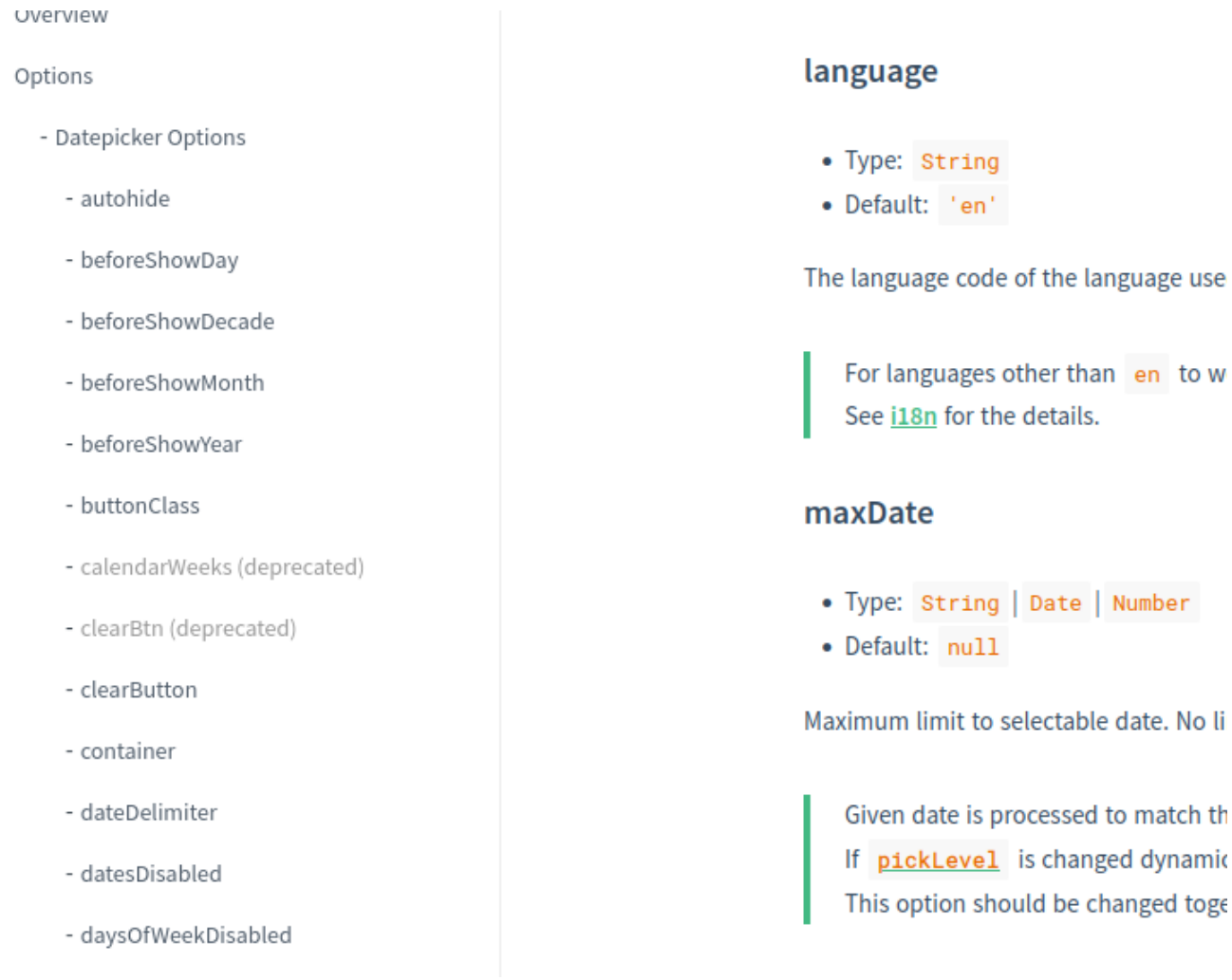
services.yaml > ...

```
parameters:
  config_directory: '%kernel.project_dir%/config/config.json'
  accomodations_directory: '%kernel.project_dir%/public/uploads/accomodations'
```

```
2 references | 0 overrides
public function __construct(ParameterBagInterface $parameterBag)
{
    $this->configFilePath = $parameterBag->get('config_directory');
}

// ! --------------- CONFIG
```

Extrait de la documentation vanillaJS picker :



Cette documentation n'est pas celle du bundle que j'utilise, cependant elle est similaire et m'a donc permit de découvrir ses options

```
const options = {
    language: "fr",
    weekStart: 1,
    clearBtn: true,
    minDate: this.today,
};
```

Extrait de la documentation Symfony :

‘How to create your own form theme’



Utilisation dans le code : création d’un thème de formulaire personnalisé.

```
{# Modifie les label type="checkbox" #}
{%- block checkbox_label -%}
    {# {%- set label_class = label_class|def
    {{- block('form_label') -}}
{%- endblock -%}

{# Modifie les erreurs #}
{%- block form_errors -%}
    {%- if errors|length > 0 -%}
        <ul>
            {%- for error in errors -%}
                <li class="{{ error_item_cla
            {%- endfor -%}
        </ul>
    {%- endif -%}
{%- endblock form_errors -%}
```

Conclusion

Ce projet a été parsemé de nombreux défis, démarrant par la nécessité de maîtriser le framework Symfony, et accentués par les contraintes temporelles qui l'accompagnaient. Mon engagement a débuté le 17 juillet, mais des problèmes de santé ont malheureusement entravé ma pleine dévotion pendant plus d'une semaine.

En entreprenant ce projet à partir de zéro, une de mes cinq semaines initiales a été dédiée à la conception conceptuelle de l'application. La complexité de ce projet s'est avérée être un véritable défi, car j'ai investi un temps considérable pour répertorier les fonctionnalités, modéliser la base de données, définir les éléments à afficher sur chaque page, établir l'arborescence des fichiers, et bien d'autres aspects.

De plus, j'ai fait le choix d'intégrer un fichier de configuration en complément de la base de données, plutôt que d'inclure toutes les données directement dans cette dernière. Cette décision m'a amené à acquérir de nouvelles compétences pour gérer cette configuration, ce qui a inévitablement ralenti mon avancement sur le projet.

Malgré les multiples obstacles, le projet progresse favorablement et les bases solides sont désormais posées, facilitant ainsi une accélération future. La majorité des fonctionnalités chronophages et complexes ont été implémentées avec succès.

En guise de conclusion, je ressens une fierté certaine à l'idée de contribuer à ce projet qui représente à mes yeux un défi de taille. Bien que le chemin à parcourir demeure substantiel, j'ai une certitude : ce projet aboutira. Il s'agit de ma première expérience avec Symfony, et elle m'a apporté un apprentissage conséquent ainsi qu'une croissance en tant que développeur.

Lexique

Agile - Les méthodes de développement en approche agile visent la satisfaction réelle des besoins d'informatisation du client. Elles l'impliquent pendant tout le développement et permettent une grande réactivité à ses demandes.

AJAX - AJAX est l'acronyme d'Asynchronous Javascript and XML. Il permet de construire des applications Web et des sites web dynamiques interactifs sur les postes clients en se servant de différentes technologies : JavaScript, CSS, JSON, XML, DOM. La combinaison de ces technologies permet d'améliorer la maniabilité et le confort d'utilisation des applications internet riches.

Design Patern - Appelés en français patrons de conception, ils permettent de formaliser des organisations empiriques de classes d'objets destinés à élaborer des modes de conceptions transposables d'une application à l'autre. L'intérêt des Design Patterns est de pouvoir travailler sur les fonctionnalités d'une famille d'objets, éventuellement de modifier ces fonctionnalités, sans avoir à revoir le code détaillé des classes.

DOM - Document Object Model. Interface de programmation normalisée par le W3C permettant à des scripts d'examiner et de modifier le contenu du navigateur web. Les objets DOM peuvent représenter une fenêtre, un document, une phrase, un style…

Entité association - Utilisé par exemple dans la méthode Merise, le modèle entité association donne une représentation de haut niveau des données de l'entreprise, appelée « modèle conceptuel ».

Feuilles de style - Les feuilles de style en cascade, généralement appelées CSS de l'anglais Cascading Style Sheets, forment un langage informatique qui décrit la présentation des documents HTML et XML. Les standards définissant CSS sont publiés par le World Wide Web Consortium (W3C).

Framework - Appelé en français cadre d'applications, c'est un ensemble de classes d'objet, utilisables pour créer des applications informatiques. Le framework fournit au développeur des objets d'interface (bouton, menu, fenêtres, boîtes de dialogue), des objets de service (collections, conteneurs) et des objets de persistance (accès aux fichiers et aux bases de données) prêts à l'emploi. Le développeur peut donc s'appuyer sur ces classes et se concentrer sur les aspects métier de son application.

RGPD - Le Règlement général sur la protection des données (RGPD) constitue le nouveau texte de référence européen en matière de protection des données à caractère personnel. Il renforce et unifie la protection des données pour les individus au sein de l'Union Européenne.

Style Défensif - Programmer dans un style défensif consiste à écrire le code de manière à anticiper les risques d'erreur et les comportements inattendus, par exemple en contrôlant que les entrées utilisateurs sont correctes. L'absence de cette anticipation peut mener à des failles de sécurité telles que les débordements de tampon.

Trigger - Appelés en français déclencheurs, les triggers sont des ordres de déclenchement d'opérations consécutifs à un événement survenant sur une table. L'intérêt des triggers est de pouvoir associer l'exécution d'une fonction en réponse à un événement, tel que la création ou la suppression d'un item survenant dans une table d'une base de données relationnelles. Ils sont utilisés pour assurer la cohérence des données dans la base, en matérialisant des contraintes qui doivent porter sur plusieurs tables.

XML - L'eXtensible Markup Language est un métalangage informatique de balisage générique. Cette syntaxe est dite « extensible » car elle permet de définir différents langages avec chacun leur vocabulaire et leur grammaire.

XSS - Le cross-site scripting est un type de faille de sécurité des sites web permettant d'injecter du contenu dans une page, permettant ainsi de provoquer des actions sur les navigateurs web visitant la page